European Journal of Operational Research 318 (2024) 927–953

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/eor

Analytics, computational intelligence and information management

Collusion by mistake: Does algorithmic sophistication drive supra-competitive profits?

Ibrahim Abada [a,*], Xavier Lambin [b,c], Nikolay Tchakarov [b]

[a] Grenoble Ecole de Management, F-38000 Grenoble, France
[b] ESSEC Business School, 3 Av. Bernard Hirsch, B.P. 50105, Cergy, 95021, France
[c] THEMA, 3 Av. Bernard Hirsch, B.P. 50105, Cergy, 95021, France

ARTICLE INFO

Keywords:
Algorithmic decision-making
Delegated decisions
Machine learning
Multi-agent reinforcement learning
Tacit collusion

ABSTRACT

A burgeoning literature shows that self-learning algorithms may, under some conditions, reach seemingly-collusive outcomes: after repeated interaction, competing algorithms earn supra-competitive profits, at the expense of efficiency and consumer welfare. This paper offers evidence that such behavior can stem from insufficient exploration during the learning process and that algorithmic sophistication might increase competition. In particular, we show that allowing for more thorough exploration does lead otherwise seemingly-collusive Q-learning algorithms to play more competitively. We first provide a theoretical illustration of this phenomenon by analyzing the competition between two stylized Q-learning algorithms in a Prisoner's Dilemma framework. Second, via simulations, we show that some more sophisticated algorithms exploit the seemingly-collusive ones. Following these results, we argue that the advancement of algorithms in sophistication and computational capabilities may, in some situations, provide a solution to the challenge of algorithmic seeming collusion, rather than exacerbate it.

1. Introduction

The advent of machine learning has been overwhelming in many fields. The availability of data, combined with an unprecedented growth in computational power, and the increasing sophistication of algorithms has rendered machine learning based decision-making pervasive: banks' lending policies, facial recognition, autonomous driving, dynamic pricing in e-market platforms, to name but a few (see for instance Chen, Guo, and Zhao (2021), Burka, Puppe, Szepesváry, and Tasnádi (2022), Dieter, Caron, and Schryen (2023), Sigrist and Leuenberger (2023), Chen, Calabrese, and Martin-Barragan (2024), Corredera and Ruiz (2023), Xiong et al. (2024), Yilmaz and Büyüktahtakın (2024), Müller, Müller, Kress, and Pesch (2022), Katsafados, Leledakis, Pyrgiotakis, Androutsopoulos, and Fergadiotis (2024), and McHale and Holmes (2023) for some recent applications). Even sophisticated home appliances (batteries, heating devices, etc.) now use machine-learning algorithms to take autonomous consumption decisions based on market signals.

Despite the indisputable efficiency gain brought about by these algorithms, the possibility of socially-undesirable behaviors, such as algorithmic-driven discriminatory biases, has been extensively reported in the literature (Dressel and Farid (2021), Hoffmann (2019), Tsamados et al. (2021), Panch, Mattie, and Atun (2019), Cowgill and Tucker (2019), Gillis and Spiess (2019), and Baker and Hawn (2022)). Another major concern is that when simultaneously trained, some Artificial Intelligence (AI) algorithms can reach outcomes that resemble collusion (a cooperative strategy through which agents agree to restrain trade in a market), even in the absence of any form of communication between them. Because of its detrimental impact on social welfare and consumers, such a phenomenon could warrant a regulation of algorithms in trading activities. In a recent and notable contribution, Calvano, Calzolari, Denicolò, and Pastorello (2020) analyze the interaction of Q-learning algorithms and observe that industry profits are much beyond the competitive ones. A related analysis is conducted in Klein (2021) for price setting Q-learning agents in a sequential game, which draws similar conclusions. In that vein, Meylahn and den Boer (2022) have recently shown that in a pricing duopoly, algorithmic cooperation is facilitated by sharing demand information between algorithms or by allowing agents to use the same pricing algorithms. Although a possible tendency towards anti-competitive behavior has now been widely documented, the mechanisms of this tendency are still not well understood. Improving this understanding would reassure consumers and facilitate the acceptation and integration of algorithms in decision-making processes. In particular, a natural reasoning is that if simple AI algorithms such as Q-learning can cooperate, then the increasing

* Corresponding author.
E-mail address: ibrahim.abada@grenoble-em.com (I. Abada).

https://doi.org/10.1016/j.ejor.2024.06.006
Received 18 February 2023; Accepted 5 June 2024
Available online 22 June 2024
0377-2217/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

sophistication of machine learning is likely to worsen the situation for consumers. An example of this reasoning can be found in Calvano, Calzolari, Denicolò, Harrington, and Pastorello (2020), page 2: "*The enhanced sophistication of learning algorithms makes it more likely that AI systems will discover profit-enhancing collusive pricing rules*". This would make collusion easier to sustain while simultaneously rendering it more challenging to detect and regulate.

This paper challenges this reasoning. We claim – and numerically verify – that the emergence of seemingly-collusive results may not be primarily influenced by algorithmic sophistication. Instead, we show that, at least in some situations, it hinges on the exploration strategy employed by algorithms (specifically, the rate at which they decrease their exploration). This confirms an intuition already suggested in Abada and Lambin (2023); this intuition was, however, built on a different and much more complex setting that precluded a clear causal inference, as well as any theoretical analysis. Using a series of numerical experiments with Q-learning, we were able to test various learning strategies and verify robustly that seemingly-collusive behavior could be limited if the exploration policy is sufficiently thorough.

The link between imperfect exploration and algorithmic cooperation has already been studied in the game theory, operations research, and AI literature (see Section 2 for a review) but we believe that it has not been thoroughly investigated through the lens of algorithmic collusion. Furthermore, to the best of our knowledge, the related theoretical works assume fixed algorithmic exploration during learning (most of the time following an ϵ-greedy procedure). In realistic applications of Q-learning though, the exploration rate is time-varying as it is progressively decreased with training to accelerate convergence, which might complicate the understanding of the relationship between the learning phase of algorithms and their propensity to cooperate after convergence.

The first contribution of our paper (Section 3) lies in the elaboration of a simple theoretical illustration of the impact of imperfect learning on cooperation *when the exploration policy changes during training*. By theoretically analyzing the interaction between two stylized Q-learners facing a Prisoner's Dilemma, we derive sufficient conditions that result in cooperation after learning in the absence of communication. These conditions have a direct interpretation in terms of exploration policy, creating a natural link with, and a theoretical illustration of, our experimental results. Using this stylized framework, we illustrate how seeming collusion can result from what we call a *mirage effect*: when their exploration rates decrease too rapidly, the algorithms remain trapped in a seemingly-collusive path. They burn-in a belief that cooperation outperforms competition and as learning carries over, it gets more and more unlikely that they will infer that competition is in fact the true individually optimal action. Because it builds on the functioning of simple and basic Q-learning algorithms competing in a stylized environment, this illustration serves the purpose of laying out some assumptions about the decrease in exploration rate that could lead to algorithmic cooperation. These assumptions are subsequently empirically verified *via* simulations on a more complex and realistic competitive environment and with more realistic algorithms. We believe that this contribution sheds an interesting light on the current debate regarding how algorithms may sustain high prices or collude.

One of the main drawbacks of the existing literature on algorithmic collusion is the systematic use of simple reinforcement learning algorithms, namely Q-learning (and recently deep Q-learning). Reinforcement learning algorithms are appealing as they can be efficiently trained off-the-job before being operated in real markets, where they can still improve their learning without significant impact on their efficiency. They do not require any prior knowledge about the system in which they evolve as they grasp the transition rules between the states of the world during learning. Some recent applications can be found in Alfonso-Sánchez, Solano, Correa-Bahnsen, Sendova, and Bravo (2024), Jin, Cui, Bai, and Qu (2024), Xu, Liu, Zhao, and Wang (2024), and Dehaybe, Catanzaro, and Chevalier (2024). Owing to its simplicity, Q-learning makes the theoretical analysis of algorithmic cooperation tractable. However, in the race to increasing efficiency, the high penetration of deep learning in many fields makes it improbable that basic Q-learning will be the dominating reinforcement learning AI in the near future. Therefore, it might not be relevant to keep studying algorithmic collusion through the lens of Q-learning only. Going in that direction, den Boer, Meylahn, and Schinkel (2022), Hettich (2021), and Asker, Fershtman, and Pakes (2022) are, to our knowledge, the only papers to analyze cooperation via more complex algorithms. Hettich (2021) and Asker et al. (2022) resort to more sophisticated, but still strongly related algorithms than tabular Q-learning, namely Deep Q-learning (DQN) for the former and synchronous Q-learning for the latter. By analyzing a market setting similar to Calvano, Calzolari, Denicolò, and Pastorello (2020), Hettich (2021) generalizes their results by reporting seemingly-collusive outcomes under standard exploration and learning rates. However, in this paper again, the inner drivers of collusion are not entirely exposed. In Asker et al. (2022), algorithmic competition is analyzed via Q-learning but with a more sophisticated *synchronous* learning process where the algorithms are assumed to know the demand function (or at least its slope) in order to simulate the profit they would earn for each possible action. The outcome of such an interaction is competitive. In den Boer et al. (2022), the authors show that Q-learning performs poorly against a reasonable alternative (the Exp3 algorithm).

The second and major contribution of our paper (Section 4) lies in broadening the scope of the analysis of algorithmic collusion by investigating the equilibrium play between reinforcement learning algorithms that are based on a different exploration policy: Continuous Actor–Critic or CAC algorithms, the Reinforce, and the Exp3 algorithms. CAC and Reinforce are notoriously more sophisticated than Q-learning and have an endogenous exploration policy. Our results suggest that the occurrence of seeming collusion in our simple setting is inherent to the specificity of Q-learning and its exploration policy: we observe that the other algorithms, that embed different exploration policies, systematically converge to the Nash equilibrium. This result is in line with the theoretical and numerical findings of Asker et al. (2022) who prove that competing (more sophisticated) *synchronous* Q-learning algorithms do not cooperate. Our approach, however, has the advantage of not having to endow the algorithms with a model of their environment. In addition, by numerically analyzing the competition between (seemingly-collusive) Q-learning and the other algorithms, we observe that these consistently outperform Q-learning, when trained in a duel. Depending on the sequentiality of the agents' choice of an algorithm, this implies, in particular, that an agent may not always have any individual incentive to deliberately choose seemingly-collusive algorithms (such as less-sophisticated tabular Q-learning). This observation should help regulators to detect collusion, as we show that the selection of a seemingly-collusive algorithm by managers may be, in some cases, a collusive decision.

Our work has several policy implications. First, we find a silver lining for regulators, as we show that the expected increase in algorithmic sophistication may actually discipline agents towards competition, increase consumer welfare, and eventually facilitate the social acceptance of AI decision-making. In particular, we conclude that regulators should insist that algorithms use thorough exploration processes. Second, we note that a simultaneous and recurring choice of seemingly-collusive algorithms by managers or traders may not be individually rational, absent collusive motivations. Therefore, the implementation of these (socially undesirable) algorithms may require a formal or tacit collusive agreement between managers. This means that when managers can choose over several algorithmic technologies with varying degrees of sophistication and tendency towards cooperative practices, existing antitrust regulations and their accompanying jurisprudence can be applied effectively. This might not always be sufficient, however, as we find that managers choosing seemingly-collusive algorithms can, in some cases, also be an equilibrium of the algorithmic selection game.

Therefore, we also recommend regulators to develop adequate tools to incentivize managers to use sophisticated algorithms when available to limit anticompetitive play. Finally, we encourage researchers involved in the study and regulation of algorithmic "collusion" to consider the full range of existing reinforcement learning algorithms, as their impact on competition and consumers may be less definite than suggested by the results available in the existing literature.

The remainder of this paper is as follows: Section 2 relates the present paper to the literature on algorithmic cooperation in Operations Research and Economics. Section 3 reports on our first contribution: it discusses the issue of cooperation in the context of Q-learning from a theoretical perspective in a simple and stylized framework. It illustrates how cooperation may originate in imperfect learning when the exploration rate decreases too rapidly. Section 4 presents our second contribution by showing a set of numerical experiments in a realistic environment. The environment is presented in Section 4.1. The case of Q-learning, which is treated in Section 4.2, confirms the results and intuitions obtained from our theoretical illustration. The cases of CAC, Reinforce, and Exp3 competition are conducted in Section 4.3 and show that more sophisticated algorithms reach competitive outcomes. We then extend our numerical protocol to analyze the competition between sophisticated algorithms and Q-learning in Section 4.4, and show that sophisticated algorithms consistently outperform Q-learning. Section 5 provides concrete policy recommendations based on our findings and Section 6 concludes the paper.

2. Literature review

A rich theoretical literature on repeated games described that collusive outcomes can be an equilibrium in a large set of games (see e.g. Mailath, Samuelson, et al. (2006), Miklós-Thal and Tucker (2019), and Hansen, Misra, and Pai (2021)). While empirical evidence of collusion has been widely documented (see Levenstein and Suslow (2012) and Connor (2003)), until recently it was unclear whether algorithms could autonomously converge to supra-competitive outcomes. Our paper first confirms the observational results of a recent but booming economic literature on AI collusion. Our interpretation of these results, however, is fundamentally different from that of other contributions. In a seminal paper, Waltman and Kaymak (2008) showed that simple Q-learning algorithms may reach seemingly-collusive outcomes: after repeated unsupervised interactions, they converge to high prices or low quantities, which increases joint profits beyond the competitive ones. The tendency of some algorithms towards sustaining supra-competitive prices, without the use of traditional collusive strategies, has been described in Brown and MacKay (2021), Meylahn and den Boer (2022), and Hansen et al. (2021) among others. Empirical works such as Brown and MacKay (2021) and Assad, Clark, Ershov, and Xu (2024) document that algorithmic price-setting is correlated with higher prices. (Because algorithmic price setting is rarely observed directly, authors analyze patterns of the frequency and timing of price updates to identify algorithmic price-setting, in the spirit of Chen, Mislove, and Wilson (2016).) Extending Waltman and Kaymak (2008), Calvano, Calzolari, Denicolò, and Pastorello (2020) and Klein (2021) show with simulations that Q-learning algorithms may reach final policies observationally similar to "reward-punishments", whereby each algorithm strategically commits to credibly punish pro-competitive deviations by the other algorithms. This, in turn, would allow algorithms to sustain collusion. This interpretation, which is relayed in many scientific works (Bramante, Calvano, Calzolari, & Shaefer, 2022; Dolgopolov, 2024; Hettich, 2021) as well as in policy discussions (Autorité de la Concurrence & Bundeskartellamt, 2019; Concorrència, 2019) is of key policy relevance and also a matter of debate (Asker et al. (2022), den Boer et al. (2022), Abada and Lambin (2023), and Epivent and Lambin (2024)). In the present paper, we confirm that the extremely simple algorithms used in the economic literature so far (basic Q-learning), tend to reach seemingly-collusive outcomes. In contrast with the recent literature, however, we suggest that seeming collusion may, at least in some situations, be due to a lack, rather than an excess in algorithmic sophistication.

Second, our paper offers one causal explanation for seeming collusion. Indeed, there is overwhelming evidence that a necessary condition for AI-driven convergence towards supra-competitive outcomes is either (1) modifying the objective function so as to internalize (at least part of) the other agents' profits (Loots & denBoer, 2023) or (2) restraining learning. Our paper investigates the second condition and shows that excessively parsimonious exploration is enough to induce seeming collusion. In that vein, the seminal papers of Littman (1994) and Hu, Wellman, et al. (1998) show that, under infinite exploration, Q-learning agents converge to a Nash equilibrium if it exists. This result is confirmed in related settings by Claus and Boutilier (1998), Chapman, Leslie, Rogers, and Jennings (2013), or Wang and Pavel (2014). With a specific application to Q-learning with a one-step memory, den Boer et al. (2022) and Meylahn and Janssen (2022) show that several equilibria may coexist, though the equilibrium selection problem is itself intricate. Some results from the game theory literature (see Cooper, de Mello, and Kleywegt (2015) for a review of the relevant results) show that supra-competitive profits may result from imperfect exploration and parameter initialization. For further detail about these results, the reader is referred to Abada and Lambin (2023). Dolgopolov (2024) uses evolutionary game theory to provide some closed-form formulations of the convergence point of two Q-learners with no memory facing a Prisoner's Dilemma with no time discounting. The algorithms may reach cooperation when exploration decreases to 0 at the limit and when they have high learning rates. Banchio and Mantegazza (2022) analyzed the equilibrium play between learning algorithms using a continuous time approximation leading to a formulation of a system of differential equations. When applied to the Prisoner's Dilemma, the authors prove that cooperation can be driven by simultaneous coupling of decisions when algorithms are symmetric. In line with our findings, the authors establish a connection between the coordination bias and the exploration rate in the case of Q-learning. However, the key mechanism is different: cycles arise from simultaneous coupling with continued (partial) exploration, while our mechanism requires exploration to decrease to 0 over time.

These insights also mirror some of the results of the data-science literature. Close to our setting, Banerjee and Sen (2007) show that stylized Q-learning agents in a simple Prisoner's Dilemma setting always reach cooperation. Importantly, the proof relies again on the assumption that the exploration process is sufficiently constrained. We observe that despite the variety of the theoretical works linking cooperation to the quality of exploration during learning, the literature has not yet treated the case of a time-varying algorithmic exploration rate which, ironically, is what is commonly performed in concrete applications. Because we attribute supra-competitive profits to possible imperfect exploration, our paper reconciles the theoretical results of the computer science literature with the experimental ones of the economic literature (Calvano, Calzolari, Denicolò, & Pastorello, 2020; Klein, 2021). Specifically, we stress the need to verify that seemingly-collusive outcomes in Q-learning are not a consequence of imperfect exploration, prior to analyzing possible punishment strategies adopted by the algorithms and concluding on the presence of "collusion". Our results find an interesting echo in some of the results of Crandall et al. (2017, 2018) that suggest that simple algorithms have a greater tendency to cooperate than more sophisticated ones. This finding is also consistent with the results of Asker et al. (2022) who analyze the impact of a more thorough learning process on the extent of collusion with Q-learning. Consistently with this observation, the present paper sheds light on whether seemingly-collusive outcomes are necessarily a consequence of excessive sophistication (e.g. whether AI develops genuinely collusive strategies with "punishment" as suggested in Calvano, Calzolari, Denicolò, and Pastorello (2020)), or instead from a lack thereof (in our case, from a rapid decrease of exploration during learning).

3. Q-learning and supra-competitive profits through rapid exploration rate decline: a theoretical illustration

In this section, we develop a detailed example that illustrates that imperfect exploration could lead Q-learning algorithms to cooperate when trained simultaneously and when their exploration rates decrease during training. Q-learning, which is one of the most basic reinforcement learning algorithms, is presented in full detail in Appendix C.1. The main objective here is to illustrate how the dynamics of the exploration policy (via a fast decrease of the exploration rate) could lead Q-learning algorithms to cooperate after learning, without deploying reward-punishment schemes as in standard models of human collusion, a behavior to which we refer as seeming collusion. (We refer the reader to the excellent analysis of reward-punishment schemes with Q-learning provided in den Boer et al. (2022).) This effect will subsequently be numerically verified on realistic settings with Q-learning in Section 4.2 and further extended to include other algorithms, including those based on Neural Networks, in Section 4.3. However, this theoretical example being relatively simple, we obviously cannot affirm that imperfect learning is the unique driver of cooperative outcomes, nor that it is necessarily at the origin of cooperation in the previous literature dedicated to algorithmic collusion. Rather, it offers a credible explanation to supra-competitive algorithmic profits. Readers with a greater interest in the economic implications of our experimental results, rather than theory, may proceed directly to Section 4.

3.1. Setting and notation

To simplify the calculations, we restrain this illustrative example to two stylized Q-learning algorithms (sometimes referred to as the "agents") indexed by $i \in \{1,2\}$. Learning occurs in the course of N episodes, after which both algorithms switch to "operations" mode (or "play" mode) where agents always play their preferred action (the limit case where learning is infinite is discussed in Section 3.3). We consider a simple game structure of a Prisoner's Dilemma where each algorithm faces two possible choices of actions: "cooperation" which is denoted by C and "competition" (or defection) which is denoted by D. The payoffs are as follows: when both agents cooperate, they both earn α. When they both compete, they both earn β. When Algorithm 1 cooperates and Algorithm 2 competes, Algorithm 1 earns σ and Algorithm 2 earns ϕ (and vice versa). The assumptions of the Prisoner's Dilemma are as follows (the strict inequalities allow us to avoid degenerate results):

$$\phi > \alpha > \beta > \sigma \tag{1}$$

which implies that competition is the only Nash equilibrium of the stage game. In line with part of the theoretical literature (see e.g. Banerjee and Sen (2007), Claus and Boutilier (1998), and Wang and Pavel (2014)) and to simplify the calculations, we deliberately overlook the description of states in the Q-matrices with the purpose of being able to derive simple theoretical results. This setting is particularly well suited to a situation where the history of actions played by the algorithms does not modify the payoff structure of the stage game — thereby making instantaneous payoffs independent from the previous states. In that regard, because a genuinely collusive behavior is usually sustained by the threat of a punishment phase in case of deviation, which, translated to algorithms, might require to keep some memory in the learning process, the main advantage of this approach is that any cooperation obtained in our setting can hardly be qualified as genuine collusion. The purpose of this example is hence *not* to analyze reward/punishment schemes as in standard models of human collusion. It is rather to exhibit other conditions where cooperative outcomes can be observed after convergence, which we will be able to tie to a rapid decrease of the exploration rate during learning. This link between seeming collusion and a rapid decrease of exploration will subsequently be confirmed numerically in Section 4 where we consider even the most general case of some reinforcement learning algorithms (including Q-learning) that incorporate a representation of states with memory.

Each agent keeps track of its long-term discounted profit in its Q-matrix. We denote by $\Pi_n^i(x)$ the value of Algorithm i's Q-matrix at Iteration n when it plays action $x \in \{C, D\}$. (Iterations are sometimes referred to as Periods.) Both Q-matrices are set at the same value at Iteration $n = -1$. The algorithms carry out learning independently as follows: Algorithm i will cooperate (play action C) with (an iteration-dependent) probability $p_i(n) \in [0,1]$ and will compete (play action D) with probability $1 - p_i(n)$. We define the exploration rate of an algorithm as the probability to play the non-optimal action according to its Q-matrix, and we denote it by $\epsilon_i(n)$ ($i \in \{1,2\}$). As an example, if at Iteration n, the Q-matrix of Algorithm 1 shows that cooperation yields higher profits than competition ($\Pi_n^1(C) > \Pi_n^1(D)$), then $\epsilon_1(n)$ is the probability to compete, i.e. $\epsilon_1(n) = 1 - p_1(n)$. This definition is slightly different from what is usually considered in the data science literature where the exploration rate is the probability to play any action, including the optimal one. The main advantage of our definition is to simplify the calculations, which justifies our choice. For the time being the probabilities $p_i(n)$ are kept constant during learning: $p_i(n) = p_i$; we discuss in Section 3.3 how our results adapt to the general (and more realistic) case where probabilities of actions evolve during learning.

We now describe the updating process of the Q-matrices during learning following the standard procedure described in Appendix C.1, where the learning rates and discount factors are denoted by $1 - \lambda_i \in (0,1)$ and $\delta_i \in (0,1)$, $i \in \{1,2\}$, respectively. The values of the Q-matrix for Algorithm i and action C at Iteration $n+1$ can be calculated by enumerating the joint actions of the algorithms. Four situations have then to be considered (index j will denote the other algorithm):

- When both i and j cooperate, which happens with probability $p_i p_j$: Algorithm i earns α and the new expected payoff is updated as follows:

$$\lambda_i \Pi_n^i(C) + (1-\lambda_i)\left(\alpha + \delta_i \mathrm{Max}_x \Pi_n^i(x)\right). \tag{2}$$

- When i cooperates and j competes, which happens with probability $p_i(1-p_j)$: Algorithm i earns σ and the new expected payoff is updated as follows:

$$\lambda_i \Pi_n^i(C) + (1-\lambda_i)\left(\sigma + \delta_i \mathrm{Max}_x \Pi_n^i(x)\right). \tag{3}$$

- When i competes and j either cooperates or competes, which happens with probability (1-p_i): Algorithm i does not update its matrix for action C (cooperation).

Therefore, the expected value of $\Pi_{n+1}^i(C)$ is ($\langle . \rangle$ denotes the expectation operator):

$$\begin{aligned}\left\langle \Pi_{n+1}^i(C)\right\rangle = (1-p_i)\left\langle \Pi_n^i(C)\right\rangle + p_i \Big[\lambda_i \left\langle \Pi_n^i(C)\right\rangle \\ + (1-\lambda_i)\left(p_j\alpha + (1-p_j)\sigma\right) + (1-\lambda_i)\delta_i \left\langle \mathrm{Max}_x \Pi_n^i(x)\right\rangle \Big]\end{aligned} \tag{4}$$

and a similar calculation can be carried out for action D (competition) to obtain the following recursive relationship:

$$\begin{aligned}\left\langle \Pi_{n+1}^i(D)\right\rangle = p_i \left\langle \Pi_n^i(D)\right\rangle + (1-p_i) \Big[\lambda_i \left\langle \Pi_n^i(D)\right\rangle \\ + (1-\lambda_i)\left(p_j\phi + (1-p_j)\beta\right) + (1-\lambda_i)\delta_i \left\langle \mathrm{Max}_x \Pi_n^i(x)\right\rangle \Big].\end{aligned} \tag{5}$$

In the operations mode (when $n \geq N$), the algorithms only play their optimal actions (according to their Q-matrices). Importantly, they continue to update their Q-matrices. The main objective now is to exhibit conditions under which algorithms will converge to the cooperative outcome. These conditions will subsequently be interpreted in terms of insufficient exploration during learning if the exploration rate decreases too fast. Our main results can be summarized as follows:

- Theorem 1 is a standard and already known benchmark result (see Hu et al. (1998) for instance). It shows that when p_1 and p_2 are constant and positive and when the learning horizon N is infinite (and consequently when all actions are visited infinitely often), algorithms can never learn that cooperation generates higher profits than competition on average at the equilibrium.
- Theorem 2 shows that when p_1 and p_2 are constant and positive but when the learning horizon N is finite, cooperation can become more profitable than competition on average after convergence (depending on the initial actions played by the algorithms and under some other technical assumptions), provided that the learning horizon N does not exceed a threshold N_{min}, a situation that we qualify as imperfect learning.
- Theorem 3 shows that threshold N_{min}, above which algorithms learn to compete on average, increases with probabilities p_1 and p_2 (recall that these are the probabilities to play cooperation at each iteration). This gives an *intuition* on how, in concrete applications of Q-learning where the probability to play the dominant action increases with iterations, algorithms can be *trapped* into believing that cooperation is optimal on average (depending on the initial actions played and under some other technical assumptions): if at some point the algorithms find that cooperation dominates competition in the Q-matrices, they will increase p_1 and p_2 in the coming iterations, raising thereby the threshold N_{min} that the learning horizon has to exceed to learn that they are better off on average if they do not cooperate. This in turn can make it impossible for the algorithms to escape the trap of believing that cooperation is more profitable when the increase of p_1 or p_2 is too fast (or equivalently, if the decrease of the exploration rate is too strong since we define this rate as the probability to play the non-optimal action, here competition: $\epsilon_i = 1 - p_i, i \in \{1,2\}$). We call this phenomenon the *mirage effect*.
- Theorem 4 and its Corollary 1 are our main theoretical results. They formalize the intuition provided by Theorem 3 and prove that in realistic applications where the exploration rate decreases with iterations, an algorithm that finds that cooperation is optimal can trap the other algorithm into learning that cooperation is optimal too on average, provided that its exploration rate decreases too fast. This phenomenon happens even when the learning horizon is infinite.

All proofs are reported in Appendix A. Unfortunately, we could not obtain closed-form results for the general algorithmic interaction we described above. Therefore, for theoretical results only, we had to simplify the scope of our analysis by considering a simpler and slightly different version of our Q-learning algorithms. We do so by adopting a mean-field approximation of the updating process of the Q-matrices. More precisely, we assume that algorithms use the expectation of their Q-matrices at Iteration n in their calculation of the new values of the Q-matrices at Iteration $n+1$. This mimics some numerical methodologies where a group of experiments are run in a batch via Monte-Carlo like simulations, and outcomes are averaged at each iteration before moving to the next one. More precisely, we replace Eqs. (4) and (5) by the following ones:

$$\left\langle \Pi^i_{n+1}(C)\right\rangle = (1-p_i)\left\langle \Pi^i_n(C)\right\rangle + p_i\Big[\lambda_i\left\langle \Pi^i_n(C)\right\rangle + (1-\lambda_i)\left(p_j\alpha + (1-p_j)\sigma\right) + (1-\lambda_i)\delta_i \mathrm{Max}_x\left\langle \Pi^i_n(x)\right\rangle\Big] \tag{6}$$

and

$$\left\langle \Pi^i_{n+1}(D)\right\rangle = p_i\left\langle \Pi^i_n(D)\right\rangle + (1-p_i)\Big[\lambda_i\left\langle \Pi^i_n(D)\right\rangle + (1-\lambda_i)\left(p_j\phi + (1-p_j)\beta\right) + (1-\lambda_i)\delta_i \mathrm{Max}_x\left\langle \Pi^i_n(x)\right\rangle\Big]. \tag{7}$$

where the only difference lies in the last term, which is the maximum of expectations instead of the expectation of the maxima. Similarly, we also assume that during the operations phase (once learning is over), algorithms play their greedy action based on the expectation of the Q-matrices. We conduct a thorough theoretical analysis of the dynamics of the expected values of the Q-matrices. Results on the distribution of the actions played by the algorithms are obtained via numerical simulations, which we report along with our theorems. Overall, this simplification allows us to unveil and explain a mechanism (the mirage effect) that relates algorithmic cooperation to the quality of exploration during the learning phase. In Section 4, we verify through a set of numerical simulations that this mechanism also exists in the most general case of the Q-learning algorithm and of other more sophisticated reinforcement learning algorithms.

Theorem 1. *When the learning horizon N is infinite and probabilities p_1 and p_2 are constant and positive, algorithms can never learn that cooperation dominates competition in expectation.*

This result is a consequence of the fact that when actions are visited infinitely often, algorithms learn the true Nash equilibrium of the stage game. A direct consequence is that the probability that algorithms cooperate indefinitely is zero in this case.

Proof. Assume that $\exists N' \in \mathbb{N}$ such that $\forall n \geq N'$, $\left\langle \Pi^1_n(C)\right\rangle > \left\langle \Pi^1_n(D)\right\rangle$, one can easily show the convergence of both sequences in (6)–(7) and calculate their limits as follows:

$$\left\langle \Pi^1_\infty(C)\right\rangle = \frac{p_2\alpha + (1-p_2)\sigma}{1-\delta_1} \tag{8}$$

$$\left\langle \Pi^1_\infty(D)\right\rangle = \frac{p_2\alpha + (1-p_2)\sigma}{1-\delta_1} + p_2(\phi-\alpha) + (1-p_2)(\beta-\sigma). \tag{9}$$

From relation (1) we observe that:

$$0 < \left\langle \Pi^1_\infty(C)\right\rangle < \left\langle \Pi^1_\infty(D)\right\rangle, \tag{10}$$

which implies that the initial assumption that $\left\langle \Pi^1_n(C)\right\rangle \geq \left\langle \Pi^1_n(D)\right\rangle$ cannot hold for all iterations after N': if learning is carried out long enough, algorithms will never learn that cooperation yields higher payoffs that competition on average. □

Fig. 1 graphs the evolution of the probability that Algorithm 1 (results are similar for Algorithm 2) finds cooperation more profitable than competition at the end of learning, $\mathbb{P}\left(\Pi^1_N(C) > \Pi^1_N(D)\right)$, as a function of the learning horizon N. We recall that this also measures the probability that the algorithm cooperates just after the end of learning when it shifts to the operations mode. Indeed, in the operations mode, algorithms do not explore anymore and play only their optimal action according to their Q-matrices. This function is estimated numerically using 10000 Monte-Carlo simulations for the following parameters (we verify that results hold qualitatively for other values satisfying Condition (1)): $\phi = 115$, $\alpha = 80$, $\beta = 55$, $\sigma = 15$, $\lambda_1 = \lambda_2 = 0.9$, $\delta_1 = \delta_2 = 0.97$, and $p_1 = p_2 = 0.6$. We assume that algorithms cooperate at Iteration 1 of learning. Fig. 1 has a similar interpretation as our theoretical results obtained in expectation: the longer the learning horizon, the less likely algorithms cooperate when they start their operations mode. We will discuss below how this probability evolves at the end of the operations mode.

3.2. Convergence to cooperative outcomes

In this section, we present a set of theorems that exhibit conditions under which algorithms whose learning process is incomplete may converge to cooperative outcomes. We still keep p_1 and p_2 constant in this section. Our developments will require a set of technical assumptions:

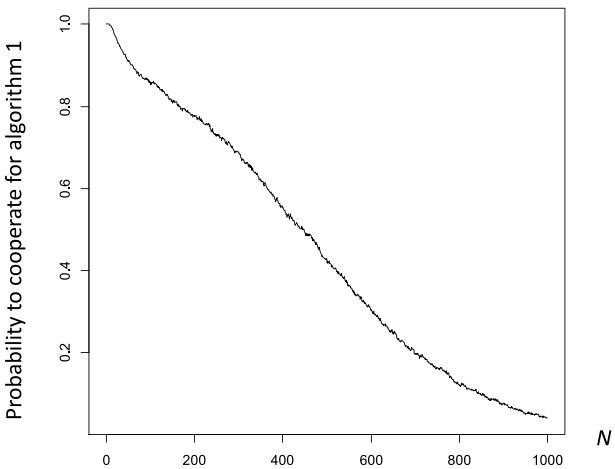

Fig. 1. The probability of cooperation at the end of learning for Algorithm 1 as a function of the learning horizon. $\phi = 115$, $\alpha = 80$, $\beta = 55$, $\sigma = 15$, $\lambda_1 = \lambda_2 = 0.9$, $\delta_1 = \delta_2 = 0.97$.

Assumption H1.

$$\Pi_0^1(D) < \Pi_0^1(C) < \mu := \frac{p_2\alpha + (1-p_2)\sigma}{1-\delta_1} \tag{11}$$

and

$$\xi\lambda_1 \geq \xi - (\Pi_0^1(C) - \Pi_0^1(D)) \tag{12}$$

with

$$\xi \equiv \left(p_1 + \delta_1 - 2\delta_1 p_1\right)\Pi_0^1(C) - (1-p_1)\Pi_0^1(D) + (1-p_1)\left(p_2\phi + (1-p_2)\beta\right) - p_1\left(p_2\alpha + (1-p_2)\sigma\right). \tag{13}$$

Assumption H1 occurs when an algorithm (here Algorithm 1) initially believes that action C generates higher payoffs than action D: $\Pi_0^1(D) < \Pi_0^1(C)$. This happens with probability 1/2 when Q-matrices are initialized randomly. The technical assumption that $\Pi_0^1(C) < \mu = \frac{p_2\alpha+(1-p_2)\sigma}{1-\delta_1}$ (which imposes that the algorithm initially believes the payoff of cooperating is less than its limit in expectation) is quite weak. It is always satisfied for values of the discount rate δ_1 close enough to 1, which holds in concrete applications of Q-learning (including the ones of the extant literature on algorithmic collusion). Condition (12) sets a constraint on the learning rate that is purely technical. It is essentially equivalent to the agent's belief that cooperation is still more effective than competition at the first iteration (and not only at Iteration 0 as per relation (11)). It can be shown that in the regime where the cooperative action is explored more extensively than defection ($1 \geq p_1 \gg 0.5$), ξ is non-positive and the assumption is always true. When $\xi \geq 0$ (which happens when $p_1 \approx 0.5$), the assumption is equivalent to:

$$\lambda_1 \geq 1 - \frac{\Pi_0^1(C) - \Pi_0^1(D)}{\xi} \tag{14}$$

which sets a lower bound on λ_1. In the numerical protocols where $\delta_1 \longrightarrow 1$ and $p_1 \approx 0.5$, it can be shown that the assumption always holds if the initial difference between the values of cooperation and defection is large enough:

$$\Pi_0^1(C) - \Pi_0^1(D) \geq \frac{\phi - \alpha + \beta - \sigma}{2}. \tag{15}$$

Theorem 2. *Assume H1 and the symmetric assumption for the second algorithm, then*

$\exists N_{min} \in \mathbb{N} > 0$ *such that if* $N \leq N_{min}$, $\forall n \leq N$, $\left\langle \Pi_n^1(D)\right\rangle \leq \left\langle \Pi_n^1(C)\right\rangle$ *and* $\left\langle \Pi_n^2(D)\right\rangle \leq \left\langle \Pi_n^2(C)\right\rangle$.

Furthermore, if in the operational phase (when $n \geq N$), algorithms play the greedy action based on the expectation of their Q-matrices, then:

If $N \leq N_{min}$, $\forall n \geq N$, $\left\langle \Pi_n^1(D)\right\rangle \leq \left\langle \Pi_n^1(C)\right\rangle$ *and* $\left\langle \Pi_n^2(D)\right\rangle \leq \left\langle \Pi_n^2(C)\right\rangle$.

All technical details of the proof of Theorem 2 can be found in Appendix A. In particular, the proof provides the expression of threshold N_{min} as follows:

$$N_{min} := \text{Min}\left(N_1, N_2\right) \tag{16}$$

where N_1 is the zero of the following function:

$$x \in \mathbb{R} \longrightarrow a_1^x\left[(\Pi_0^1(C)-\mu)(1-\rho)\right] + a_2^x\left[\rho(\Pi_0^1(C)-\mu) - (\Pi_0^1(D)-\nu)\right] + (\mu-\nu)$$

with

$$a_1 = p_1\lambda_1 + (1-p_1) + (1-\lambda_1)\delta_1 p_1, \tag{17}$$

$$a_2 = (1-p_1)\lambda_1 + p_1, \tag{18}$$

$$\mu = \frac{p_2\alpha + (1-p_2)\sigma}{1-\delta_1}, \tag{19}$$

$$\nu = \mu + p_2(\phi - \alpha) + (1-p_2)(\beta - \sigma), \tag{20}$$

$$\rho = \frac{c_2}{a_1 - a_2} = \frac{\delta_1(1-p_1)}{1 - 2p_1 + \delta_1 p_1}. \tag{21}$$

We demonstrate in the proof that the zero exists and is unique. The expression is symmetric for N_2 and can be obtained by replacing Algorithm 1's parameters by those of Algorithm 2. Recall that N is the learning horizon (or duration). The intuition behind Theorem 2 is as follows: if algorithms happen to believe at some point that cooperation outperforms competition, this belief will carry over in expectation during learning until a threshold N_{min} is exceeded. If the learning horizon N remains below N_{min}, this belief will persist also during operations (once learning is over) for both algorithms (recall that algorithms still update their Q-matrices even when they do not explore anymore). In other words, if the exploration phase is not long enough, algorithms can get *trapped* into believing that cooperation is superior to competition on average. This is in clear contradiction with the Nash equilibrium of the one-period Prisoner's Dilemma game, and that of the repeated interaction under infinite learning found in Theorem 1. With Monte-Carlo simulations, Fig. 2 illustrates what this result (obtained in expectation) implies for the distribution of the actions taken by the algorithms at the end of the operations mode. More precisely, we consider the same data as in Fig. 1 and we report on the probability that Algorithm 1 (results are similar for Algorithm 2) cooperates at Iteration $n = 2N$, $\mathbb{P}\left(\Pi_{2N}^1(C) > \Pi_{2N}^1(D)\right)$, for various levels of the learning duration N. The figure also graphs the evolution of $\left\langle \Pi_N^1(C)\right\rangle - \left\langle \Pi_N^1(D)\right\rangle$ with respect to N. The vertical line represents threshold N_{min}. The interpretation is here again, straightforward: the longer algorithms learn, the less likely they will cooperate during the operations mode. We also observe that if the learning duration exceeds N_{min}, the probability to cooperate at the end falls below that of competition. This observation is, however, purely numerical.

In the rest of this section, we will assume that Assumption H1 —plus the symmetric one for Algorithm 2 — always holds. We now analyze the variations of threshold N_{min} (introduced in Theorem 2) with respect to parameters p_1 and p_2. We then denote this threshold by $N_{min}(p_1, p_2)$ and introduce Assumptions H2 and H3 as follows:

Assumption H2.

$$(\phi - \alpha) - (\beta - \sigma) \leq 0. \tag{22}$$

Assumption H2 has an interpretation as a symmetry premium: it ensures that both agents are on average better off if they randomly play the same action than if they randomly play different actions. We discuss some of its implications and how it can sometimes be relaxed at the end of this section.

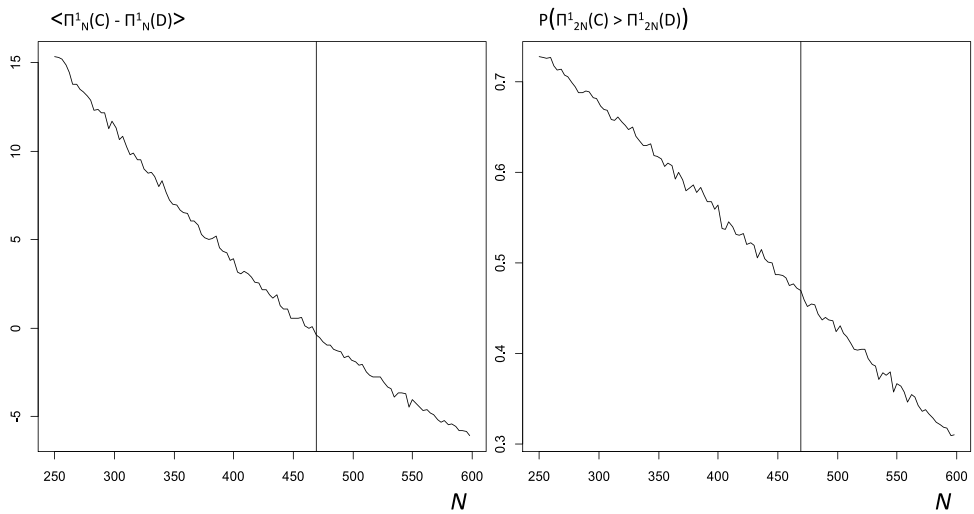

Fig. 2. The expected difference between the Q-values for cooperation and defection at the end of exploration (left) and the probability of cooperation at the end of the operations phase (right) for Algorithm 1 as a function of the learning horizon. The vertical line represents threshold N_{min}. $\phi = 115$, $\alpha = 80$, $\beta = 55$, $\sigma = 15$, $\lambda_1 = \lambda_2 = 0.9$, $\delta_1 = \delta_2 = 0.97$, $p_1 = p_2 = 0.6$.

Assumption H3.

$$p_1 \text{ and } p_2 \geq \frac{1}{2}. \tag{23}$$

Assumption H3 is natural for realistic applications of Q-learning where the exploration rates, which are $1 - p_i$ when cooperation is the algorithms' preferred action, decrease with learning. We treat this case in Section 3.3, where we will discuss the assumption.

To ease the exposition – and because we could not obtain closed-form results in the general case – we restrain some of our results and their analysis to the particular situation where the discount rate is close to 1:

Assumption H4.

$$\delta_1 \text{ and } \delta_2 \longrightarrow 1. \tag{24}$$

As mentioned above, Assumption H4 aligns well with observed or chosen values of δ in practical scenarios. It is also well known in the literature of repeated games that a higher discount rate facilitates cooperation. This is in line with our theoretical results obtained with the stylized Q-learning: as an example, we can prove that the threshold N_{min} that has to be reached during learning to dissuade algorithms from favoring cooperation over competition on average increases with higher discount rates. As a consequence, cooperation is more likely to arise with higher discount rates. Note that our mechanism is based on the persistence of an erroneous belief that cooperation yields higher payoffs than defection. In particular, this mechanism differs from the theoretical literature on repeated games, where cooperation is maintained through the threat of a long-lasting punishment, as we do not allow algorithms to observe pas actions explicitly. Hence, we show that some patterns observationally similar to collusion can be explained by imperfect exploration. We have also verified in the numerical simulations we report in Section 4 that cooperation does not occur with low discount rates. It is therefore all the more relevant to analyze algorithmic cooperation under Assumption H4 .

Theorem 3. *Assume H1 for both algorithms and H2–H3. The threshold $N_{min}(p_1, p_2)$, which has been defined in Theorem 2, is an increasing function of p_2. Under the additional Assumption H4, the threshold $N_{min}(p_1, p_2)$ is either infinite or is an increasing function of p_1.*

Theorem 3 is central to our analysis as it already gives an *intuition* on the link between a rapid decrease of exploration and algorithmic cooperation. Indeed, the theorem presents some conditions to the emergence of an interesting phenomenon during learning that will eventually *trap* algorithms into playing cooperation in the long-run. In practice, to reduce the computation time in concrete applications of Q-learning, the exploration rate, which in our case is the probability of actions that are not optimal according to the Q-table, is decreased during learning. This allows the algorithms to better optimize their operations around the actions where they believe they earn higher profits. Applied to our setting, this means that if at some point during learning, the Q-values for cooperation of both algorithms are higher than that of competition, the probability of playing competition decreases, which translates into increased values p_1 and p_2. As a consequence, the threshold N_{min} above which algorithms can learn that cooperation yields lower profits in expectation than competition increases, making defection less likely to happen. In turn, algorithms will continue to believe that cooperation is optimal in expectation and would increase p_1 and p_2 further. In other words, learning has to be sufficiently long for algorithms to deduce that the expected profit of competition can be higher than that of cooperation, which might never happen if p_1 and p_2 increase too fast even when the learning duration is infinite (or equivalently, if the decrease of the exploration rate is too strong since we define this rate as the probability to play the non-optimal action, here competition: $\epsilon_i = 1 - p_i, i \in \{1, 2\}$). We will refer to this phenomenon as the *mirage* effect. We call this effect a mirage because the closer an algorithm gets to the (dynamic) threshold where it could avoid cooperation (in expectation), the farther this threshold gets. Furthermore, the fact that the threshold of a player directly depends on the exploration rate of the other player creates some externalities that can reinforce the mirage: even if one algorithm explores extensively and during a long period of learning, its threshold might still increase with n depending on the exploration policy followed by the other algorithm, making it even harder to break the spiral of cooperation. In the next section, we provide an economic and practical interpretation of our theorems and a rigorous formalization of the mirage effect.

Taking stock of the interpretations of our results, we remark that Assumptions H1 to H4 are all sufficient but not necessary to create a mirage effect. As an example, Assumption H1 can be relaxed by analyzing other regimes of learning: e.g. when one player initially tests cooperation and the other competition. In addition, we note that some conditions used in the proof of Theorem 3 are less restrictive than Assumptions H2 and H3, which implies that they can sometimes be relaxed.

3.3. Economic interpretation: the mirage effect or imperfect learning as a driver of seeming collusion

In this section, we build on the theoretical analysis presented in Section 3.2 and extend it to the general case where exploration probabilities decrease during learning. The aim is to formally define the mirage effect and provide a sufficient condition for triggering it based on the pace of exploration decline. In order to do this, we need now to account for the fact that the exploration policy decreases with the algorithms' Iterations n by explicitly making the exploration rates $\epsilon_i(n)$, and consequently $p_i(n)$, depend on n in the system of recursive Eqs. (6) and (7) (and the symmetric ones for the second algorithm). We also analyze the extreme case where the duration of learning is infinite ($N \longrightarrow +\infty$) which makes it, in theory, more difficult for algorithms to cooperate (by Theorem 1). To ease the exposition and focus on the practical implications of our work, we relegate the full mathematical formulation and analysis of the mirage effect to Appendix A.3 and only present and discuss our main result here. We simplify further the exposition and calculations by considering that only Algorithm 2 changes its exploration rate (which implies that $\epsilon_1(n)$ is kept constant) and that it is in a mode where it believes that cooperation outperforms competition as a consequence of Assumption H1. Hence, its exploration rate $\epsilon_2(n)$ is equal to $1 - p_2(n)$ (the probability to play the non-optimal action):

$$\forall n \in \mathbb{N},\ \epsilon_2(n) = 1 - p_2(n) \in [0, 1]. \tag{25}$$

Because the exploration rate $\epsilon_2(n)$ decreases with n, we have that $p_2(n)$ increases with n. Furthermore, we classically assume that $p_2(0) = \frac{1}{2}$ (algorithms initially play both actions equally). With these considerations, Assumption H3 becomes quite natural. We denote by $p_{2,f}$ the limit of sequence $p_2(n)$ when $n \longrightarrow +\infty$ ($p_2(n)$ is an increasing sequence of n; as it is bounded, it converges) and we will require one final assumption:

Assumption H5.

$$\phi\sigma \leq \alpha\beta. \tag{26}$$

Confronted with Assumptions H2, H5 implies that the symmetry premium of the game is not too high.

Theorem 4. *Assume H1 for both algorithms and H2 to H5. If after learning, algorithms play their greedy action based on the expectation of their Q-matrices, then there exists two positive constants m and q and an integer n^* such that if $\forall n \leq n^*,\ |\frac{\partial \epsilon_2}{\partial n}(n)| \geq m$ and $\forall n \in \mathbb{N},\ |\frac{\partial \epsilon_2}{\partial n}(n)| \leq q$, then:*

$$p_1 > \frac{p_{2,f}\phi + (1 - p_{2,f})\beta}{p_{2,f}(\phi + \alpha) + (1 - p_{2,f})(\beta + \sigma)} \Longrightarrow \forall n \in \mathbb{N},\ \left\langle \Pi_n^1(C) \right\rangle - \left\langle \Pi_n^1(D) \right\rangle \geq 0. \tag{27}$$

We prove in Appendix A.6 that positive constants q and m are such that $m \leq q$ most of the time, which ensures the feasibility of the conditions of the theorem. Theorem 4 shows that when an algorithm (here Algorithm 2) starts to believe that cooperation is more profitable than competition, then when its exploration rate $\epsilon_2(n) = 1 - p_2(n)$ decreases rapidly with iterations ($\forall n \leq n^*,\ |\frac{\partial \epsilon_2}{\partial n}(n)| \geq m$), a mirage effect emerges if probability p_1 is sufficiently high: the fast learning algorithm will trap the other one into believing that cooperation is more profitable in expectation that competition, even when the learning horizon N is infinite. Condition $|\frac{\partial \epsilon_2}{\partial n}(n)| \leq q$ is a reasonable technical assumption that allows us to approximate the solutions of the general Q-learning competition problem with decreasing exploration (see Appendix A.3 for all details). From a practical perspective, as we discuss in Appendix A.6 of the Appendix, the assumption is not binding when the symmetry premium is low enough. Besides, our extensive numerical simulations indicate that the assumption is rarely necessary to generate a mirage effect.

In many applications of Q-learning, the exploration rate is progressively decreased until it reaches a limit (often close to 0). Theorem 4 can be rewritten in terms of an upper bound on the final exploration rate to induce cooperation via a mirage. Indeed, condition $p_1 > \frac{p_{2,f}\phi + (1-p_{2,f})\beta}{p_{2,f}(\phi+\alpha)+(1-p_{2,f})(\beta+\sigma)}$ can be reformulated as (it can be shown that the right-hand side of (28) is always positive under Assumption H2 and if $p_1 \geq \frac{\phi}{\phi+\alpha}$):

$$\epsilon_{2,f} < \frac{p_1(\phi + \alpha) - \phi}{p_1(\phi + \alpha - \beta - \sigma) + \beta - \phi} \tag{28}$$

where $\epsilon_{2,f}$ denotes the final exploration rate $\epsilon_{2,f} = 1 - p_{2,f}$, which leads to the following corollary:

Corollary 1. *Assume H1 for both algorithms and H2 to H5. If after learning, algorithms play their greedy action based on the expectation of their Q-matrices, then there exists two positive constants m and q and an integer n^* such that if $\forall n \leq n^*,\ |\frac{\partial \epsilon_2}{\partial n}(n)| \geq m$ and $\forall n \in \mathbb{N},\ |\frac{\partial \epsilon_2}{\partial n}(n)| \leq q$, then:*

$$\epsilon_{2,f} < \frac{p_1(\phi + \alpha) - \phi}{p_1(\phi + \alpha - \beta - \sigma) + \beta - \phi} \Longrightarrow \forall n \in \mathbb{N},\ \left\langle \Pi_n^1(C) \right\rangle - \left\langle \Pi_n^1(D) \right\rangle \geq 0. \tag{29}$$

We show in Appendix A.7 how relation (29) extends to the case when both algorithms modify their exploration rate. It turns out that it suffices that one algorithm explores thoroughly to prevent cooperation to dominate competition (in expectation). In a multi-agent framework involving many algorithms, the assumptions leading to a mirage effect must be satisfied for each player in order to generalize the results of our theorems. The natural implication is that it becomes less and less likely that a mirage effect is created (due to insufficient learning) with an increasing number of algorithms. This observation is also coherent with the empirical results of Calvano, Calzolari, Denicolò, and Pastorello (2020), Waltman and Kaymak (2008), and Abada and Lambin (2023) who observe decreasing profits when the number of Q-learning agents increase. This finding is consistent with the results of oligopoly and collusion theories. In our setting, however, imperfect learning can cause cooperation without assuming anything about the economic savviness of algorithms.

We now simulate the interaction between the two stylized Q-learning algorithms analyzed in this section in the general case where the exploration rate decreases with learning. We assume that algorithms are symmetric: in particular they both decrease their exploration at the same pace: $\forall n \in \mathbb{N},\ \epsilon_1(n) = \epsilon_2(n) = \epsilon(n)$. The initial exploration rates are set to $\epsilon(0) = 0.5$. Both algorithms learn at the same time and exploration is exponentially decreased during learning as follows:

$$\left(\epsilon(n+1) - \epsilon_f\right) = (1 - \gamma)\left(\epsilon(n) - \epsilon_f\right) \tag{30}$$

where $\epsilon_f < \epsilon(0)$ denotes the final exploration rate of both algorithms. It can be shown that relationship (30) is equivalent to:

$$\forall n \in \mathbb{N},\ \epsilon(n) = \epsilon_f + (1 - \gamma)^n \left(\epsilon(0) - \epsilon_f\right), \tag{31}$$

which implies, if $\gamma \in (0, 1)$, that the exploration rate $\epsilon(n)$ follows an exponential decrease from $\epsilon(0)$ down to ϵ_f (see Chuchro and Gupta (2017), Kormelink, Drugan, and Wiering (2018), or Beikmohammadi (2022) for some applications). Exponential decay is one of the methods used to reduce the exploration rate in practical Q-learning applications. The speed of the exploration decrease can be tuned via two parameters: $\gamma \in (0, 1)$ and the final exploration rate ϵ_f. To ease the interpretation of our numerical results, in all our simulations, we play with exploration by changing the final exploration rate and by fixing parameter γ with the purpose to interpret them in light of Corollary 1. We assume that algorithms cooperate at Iteration 0 (which satisfies Assumption H1) and we consider the same data as in Figs. 1 and 2. Fig. 3 shows the evolution of the expected Q-matrix of Algorithm 1 and its probability to cooperate (recall that algorithms are symmetric) with iterations, during learning

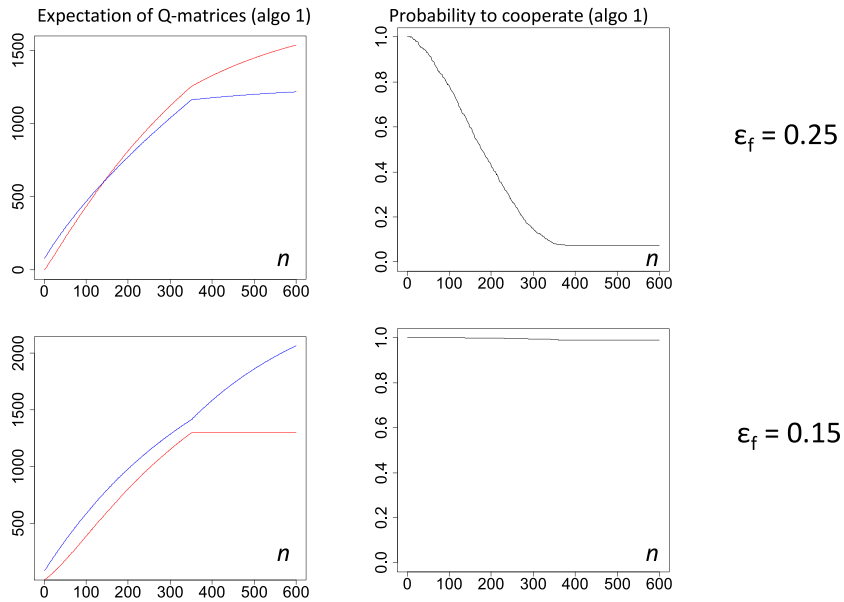

Fig. 3. Left: evolution of the expectation of the Q-values for Algorithm 1 with iterations for cooperation (blue) and competition (red) and for two final exploration rates. Right: evolution of the probability to cooperate with iterations for two final exploration rates. $\phi = 115$, $\alpha = 80$, $\beta = 55$, $\sigma = 15$, $\lambda_1 = \lambda_2 = 0.9$, $\delta_1 = \delta_2 = 0.97$. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

and operations, for $1-\gamma = 0.8$ and for two values of the final exploration rate: 0.15 and 0.25. The learning horizon N is set to 350. Extending our theoretical results, these simulations highlight how a parsimonious exploration ($\epsilon_f = 0.15$) can trap algorithms into cooperation in the long-term, when they initially "believe" that cooperation is the optimal choice of action, as the probability to cooperate is almost equal to 1. On the contrary, when exploration is more thorough ($\epsilon_f = 0.25$), the probability to cooperate decreases and algorithms manage to learn that competition is more profitable than cooperation on average. A sensitivity analysis assessing the impact of parameters of the Prisoner's Dilemma game (ϕ, α, β, and σ) on the propensity of algorithms to cooperate is provided in Appendix B.

To conclude this section and before moving on to more realistic simulations, it is important to acknowledge that while the mirage effect may play a role in seeming collusion, it is not the only factor to consider. It is possible that more sophisticated algorithms with some degree of memory during learning may employ genuinely collusive strategies by imposing targeted punishments for deviations from the collusive path, as suggested by Calvano, Calzolari, Denicolò, and Pastorello (2020). That being said, we believe that any investigation into algorithmic collusion should rule out the effects we describe–specifically, imperfect learning and mirage effects–before delving directly into the examination of collusion itself.

4. Simulation-based evidence

From our theoretical analysis set out in Section 3, we derive three testable conjectures, namely that:

- Conjecture (1): Seeming collusion may result from imperfect exploration caused by a rapid decline in the exploration rate.

From Conjecture (1), we propose a more general conjecture:

- Conjecture (2): Seeming collusion is not necessarily caused by algorithmic sophistication. Rather, increased algorithmic sophistication could mitigate such behavior by enabling more efficient exploration strategies.

Acknowledging the existence of more sophisticated (and therefore, more competitive according to Conjecture (2)) algorithms enriches the action space of a supergame where managers would first (non-cooperatively) choose the type of algorithm they implement, and then let them operate autonomously. This naturally leads us to our third conjecture:

- Conjecture (3): Providing managers with access to more sophisticated algorithms may enhance the competitiveness of strategic interactions in equilibrium.

To test these conjectures and derive clear policy implications, we first increase the realism of our agents and their economic environment. This (canonical) environment, which can be seen as a generalization of the Prisoner's Dilemma framework of Section 3, is described in Section 4.1. We then test the three conjectures successively with different algorithmic interaction experiments in Sections 4.2, 4.3, and 4.4. These experiments numerically demonstrate Conjectures (1), (2), and (3), respectively.

4.1. The experimental protocol and the environment

In the numerical simulations conducted in this section, we increase the realism of the algorithms of Section 3, as well as their economic environment. More precisely, we follow the protocol of Waltman and Kaymak (2008), who consider agents competing *Ã la* Cournot (i.e. on quantities). Cournot competition generalizes the Prisoner's Dilemma framework of Section 3 to an action space of size possibly greater than two. Here again, cooperation on monopolistic quantities cannot be a (one-shot) equilibrium of the Cournot model. The inverse demand function is piecewise linear in quantities and is kept the same during the whole duration of the algorithmic interaction:

$$p = P(q_1 + q_2) = \text{Max}(10 - q_1 - q_2, 0). \tag{32}$$

Marginal costs are normalized to 0. Therefore, if Algorithm 1 (respectively Algorithm 2) plays quantity q_1 (respectively quantity q_2), its immediate profit is $q_1\text{Max}(10 - q_1 - q_2, 0)$ (respectively $q_2\text{Max}(10 - q_1 - q_2, 0)$). Cartel and Nash profits of the stage game can be derived straightforwardly:

$$\pi^{Nash} = 22.2$$
$$\pi^{Cartel} = 25.$$

We also endow the algorithms with some memory in their representation of states. The introduction of states in the modeling of algorithms (as compared with the stylized Q-learning) should, if anything, facilitate cooperation because it may allow them to build, in theory, punishment strategies to sustain collusion, as in standard models of (human) collusion. A state observed by an algorithm is represented by the market price of the previous period. This is in line with the literature which observes cooperation between algorithms endowed with a one-step memory (see Calvano, Calzolari, Denicolò, and Pastorello (2020) and Abada and Lambin (2023)). In the case of two algorithms, the action of the other algorithm, i.e. its quantity, may be inferred from the observation of the market price, conditional on one's own action. As a consequence, the state space is the set of feasible market prices at each iteration. Q-matrices are initialized at the long-term discounted Nash profit $\frac{\pi^{Nash}}{2(1-\delta)}$. Multiple Q-matrix initializations were examined, and they exhibited no substantial impact on the final equilibrium points following the learning process. This can be attributed to the extended exploration phase we employ, which effectively leads to the rapid obsolescence of initial Q-values. In the case of Q-learning, states (the market price of the previous iteration) and actions (quantities) are discretized into 100 values ranging evenly between 0 and 10. In the base case, we set the learning rate to 0.1 and the discount factor δ to 0.95, which are common values used in the AI literature. We have numerically checked that all the results we are about to report are robust with respect to some main assumptions on hyper-parameters of Q-learning such as the learning rate (in the range [0.05-0.2]), the discount factor, and the discretization of the actions and states sets. We remind that the updating of the Q-matrices is thoroughly explained in Appendix C.1.

For each experiment, we perform 100 simulations; Appendix D.1 demonstrates that this number of simulations is sufficient to validate the accuracy of our sample mean and standard errors estimates. Each simulation consists in $N = 50{,}000$ episodes of learning after which algorithms operate optimally. We analyze the distribution of agent's payoffs accrued during the learning ("training") phase and then during the operations ("play") phase across all simulations. We then compare the outcome to the cartel and Nash benchmarks, which allows us to identify seeming collusion. To do this, we need a simple measure of seeming collusion. One way to measure the degree of algorithmic seeming collusion is to compare the joint profits of the algorithms, that we denote by π^{AI}, to π^{Cartel} and π^{Nash}. To that aim, we define the cooperation rate υ (in line with Calvano, Calzolari, Denicolò, and Pastorello (2020) and other works), as follows:

$$\upsilon = \frac{\Pi^{AI} - \Pi^{Cournot}}{\Pi^{Cartel} - \Pi^{Cournot}}. \tag{33}$$

Algorithms reach a supra-competitive (or seemingly-collusive) outcome whenever $\upsilon > 0$ implying that they generate profits higher than the Nash-Cournot profit. The higher υ, the more seemingly-collusive the algorithms. It may be negative if algorithms choose inefficient actions, resulting in joint profits lower than the Nash profits. This situation might occur for many reasons: an overly extensive exploration policy that forces algorithms to explore non-profitable actions too often during learning, overly large sets of actions/states spaces, etc. The cooperation rate has an upper bound of 1, a value that indicates a perfect seemingly-collusive algorithmic output (algorithms behave jointly like a monopoly). The exploration policy of the algorithms is explained in the next section. Except for instances where the minimum exploration level is explicitly specified, all hyper-parameters are identical for both algorithms.

4.2. Better exploration may induce more competition with Q-learning

In this section we show (numerically) that simple and autonomous Q-learning agents tend to reach seemingly-collusive outcomes. However, Q-learners that have more thorough exploration act more competitively, which provides an empirical validation of Conjecture (1).

During learning, exploration of both Q-learning algorithms is time-varying epsilon-greedy with exploration rates $\epsilon_1(n)$ and $\epsilon_2(n)$ that follow an exponential decrease from 1, down to a minimum and positive exploration rate that we denote by ϵ_f when algorithms are symmetric (otherwise we denote them by $\epsilon_{1,f}$ and $\epsilon_{2,f}$), following the dynamics described in relation (30). In a sensitivity analysis, we test the influence of $\epsilon_{1,f}$ and $\epsilon_{2,f}$ on the cooperation rate. The exponential decrease parameter $1-\gamma$ is kept constant and equal to 0.9998, which ensures that $\epsilon = \epsilon_f$ at least from period 35,000 to 50,000.

Fig. 4 displays the joint profits of symmetric agents during learning for various levels of ϵ_f: 0.1%, (blue), 1% (red) and 10% (green), as a function of the learning episode. As we expected, profits are low during early interactions as agents play mostly randomly in order to learn about their environment. After a number of periods, the actions stabilize, and so do the payoffs. Interestingly, the agents endowed with a very parsimonious exploration policy (blue line) derive strong supra-competitive profits, much beyond Nash profits (red horizontal line). As the exploration policy becomes more extensive (pink and green line), joint profits decrease. Eventually, the algorithms earn on average less than the Nash outcome when the exploration policy is too high ($\epsilon_f = 10\%$) as exploration itself becomes too costly since it forces algorithms to play non-optimal actions too often during learning.

Though informative about how algorithms learn to interact, learning is not the only phase relevant to regulators and policymakers. Indeed, it is useful to look at algorithms' interactions during their operational phase (i.e. after learning, when exogenous exploration is terminated and algorithms only follow their best policy). The cooperation rate that prevails in the operational phase is reported in Fig. 5. The horizontal lines represent the median cooperation rate for 100 trained algorithm pairs, whose operational phase is repeated 100 times with randomly chosen initial states. As a robustness check, we also add the case of non-symmetric algorithms (with different final exploration rates). Depending on the pair of exploration policies, the cooperation rate ranges from 0.2 to 0.9. This is comparable to the values found in the literature in other settings (0.3 to 0.8 in Waltman and Kaymak (2008), 0.85 in Calvano, Calzolari, Denicolò, and Pastorello (2020), and about 0.75 in Klein (2021)).

Our numerical results concur with our theoretical illustration: with Q-learning, a more thorough exploration increases competition. As an example, the cooperation rate of symmetric algorithms drops (on average) from 0.80 to 0.35 when the final exploration rate ϵ_f increases from 0.1% to 10%. Even though supra-competitive profits are not totally eliminated, these results show that extensive or slow exploration may alleviate the issue of algorithmic supra-competitive profits, while constrained exploration generates higher profits and prices. This finding is similar in flavor to that of Hansen et al. (2021) who find in the particular context of an Upper Confidence Bound (UCB) Bandit Algorithm that firms may have an incentive to be "willfully misspecified". In the rest of the paper, we will describe Q-learning as a *seemingly-collusive* algorithm.

4.3. Algorithmic sophistication may induce more competition: insights from other algorithms

In this section we study the outcome of algorithmic competition via three additional algorithms. These are:

- The Continuous Actor–Critic algorithm (CAC). Actor–Critic algorithms (Doya, 2000) are today widely used as foundational reinforcement-learning techniques in many fields: computer vision, robotics, autonomous driving, antilock braking system, etc (see Haarnoja et al. (2018), Dogru, Velswamy, and Huang (2021), and Gajjar, Khaparde, Nagaraju, and Soman (2002) for some applications). Conceptually, CAC differs from Q-learning as follows: (1) the estimation of the value function and the choice of actions to play are done via two correlated neural networks, (2) the states and actions are modeled in continuous spaces, and (3) the exploration policy is endogenous to the learning process.

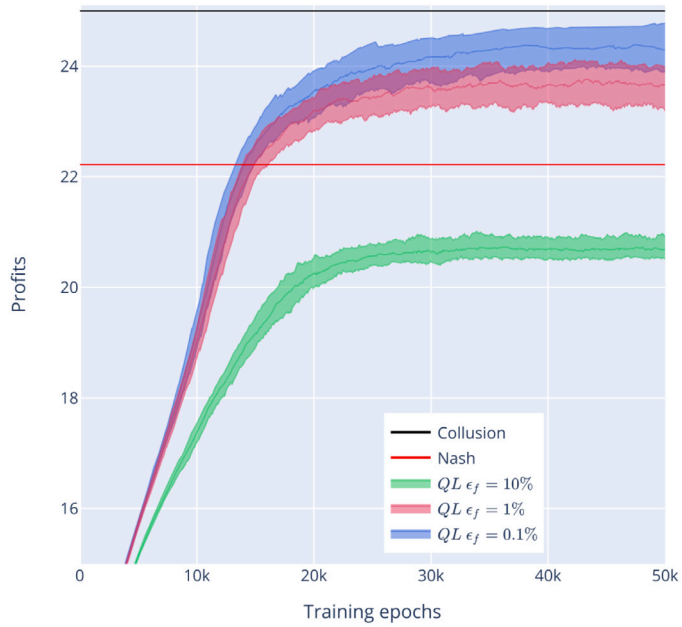

Fig. 4. Industry profits when two symmetric Q-learners compete, for parsimonious ($\epsilon_f = 0.1\%$, blue), medium ($\epsilon_f = 1\%$, red), and expansive ($\epsilon_f = 10\%$, green) exploration policy during learning. Ranges show the 10–90 percentiles of the profit distribution. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

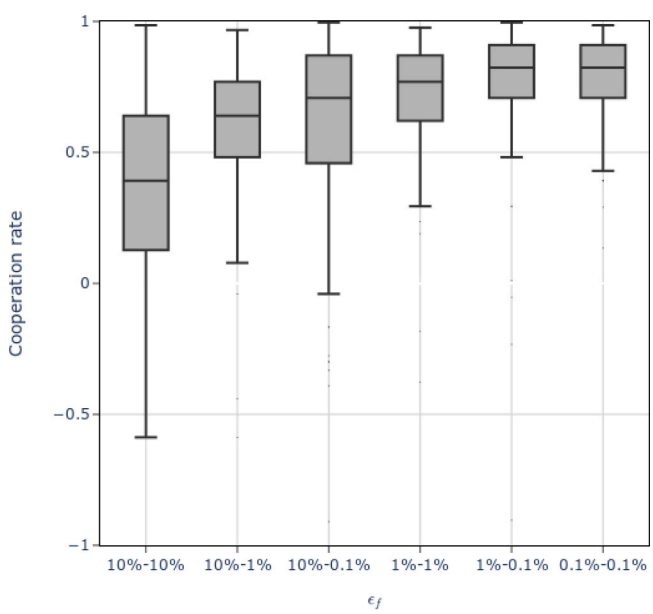

Fig. 5. Cooperation rate after learning (play mode) when two Q-learners compete, for various exploration policies. The x-axis reports the final exploration rates of each Q-learner.

- The Reinforce algorithm (Williams, 1992). This algorithm uses gradient descent methods to directly optimize the policy without necessarily estimating the value function, which constitutes its main difference with Q-learning. Its applications include robot locomotion and artificial dynamic biped walking (Tedrake, , Zhang, & Seung, 2004).
- The Exp3 algorithm (Lattimore & Szepesvári, 2020) is a basic policy optimization algorithm that has been developed to solve the multi-armed bandit problem. It has recently been applied to analyze algorithmic collusion in den Boer et al. (2022) and differs from Q-learning as follows: Exp3 does not require a representation of states of the system and it does not estimate the value function.

Q-learning is mostly used as an introduction to reinforcement learning AI. The three algorithms proposed above are natural alternatives to Q-learning, some of them (CAC and Reinforce) being widely recognized as superior to basic Q-learning in various settings. We show that CAC, Reinforce, and Exp3 algorithms quickly learn to play competitively, which provides support for our Conjecture (2). Mirroring our study of Q-learning in Section 4.2, the state observed by an algorithm (in the case of CAC and Reinforce) is the market price observed in the previous iteration. An exhaustive description of the algorithms and further technical details of implementation are provided in Appendices C.2, C.3, and C.4.

Fig. 6 reports on the joint profits when two symmetric CAC algorithms compete against each other over learning episodes (pink line). To ease the comparison with Q-learning, we also show the learning history of two competing symmetric Q-learners with $\epsilon_f = 0.1\%$ (blue line). We observe a notably faster convergence of our CAC algorithms compared to Q-learners. Furthermore, post-convergence, CAC algorithms consistently exhibit competitive play, a striking departure from the behavior of Q-learners. These results remained consistent across a broad range of reasonable hyperparameters, including variations in learning rate, discount factor, and network size. Similar outcomes were also obtained for the Reinforce and Exp3 algorithms. This result calls for caution considering the claims of some of the extant literature where it is sometimes conjectured that more sophisticated algorithms would, if anything, collude more effectively than coarser algorithms. Instead, sophistication appears to allow algorithms to learn to compete more effectively, at least for the particular set of algorithms we analyze and for the particular economic environment in which we make them compete.

Fig. 7 shows the cooperation rate during the operations phase of various algorithmic duels. The results confirm Conjecture (2): the cooperation rate is close to 0 for CAC, Reinforce, and Exp3 algorithms. We conjecture that this result may be attributed to the specificity of the exploration policy of these algorithms as contrary to Q-learning which strives to optimize the value function, all three of these algorithms are

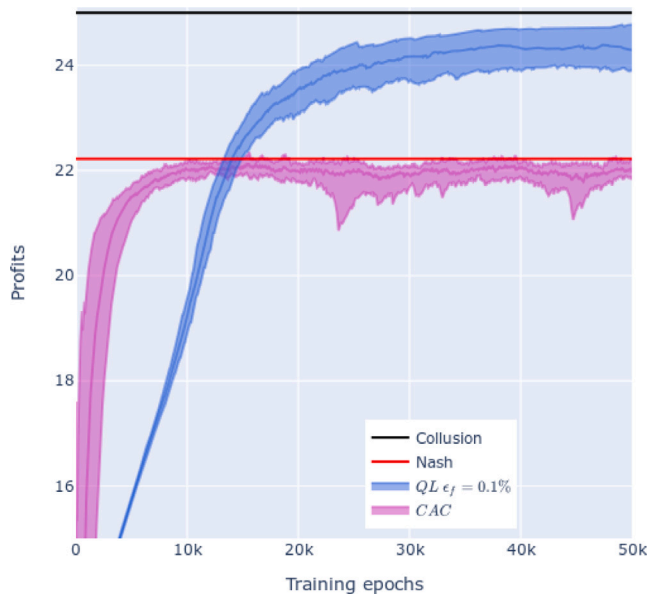

Fig. 6. Industry profits during learning when two symmetric CAC algorithms compete (purple) vs when two symmetric Q-learners compete ($\epsilon_f = 0.1\%$, blue). Ranges show the 10-90 percentiles of the profit distribution. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

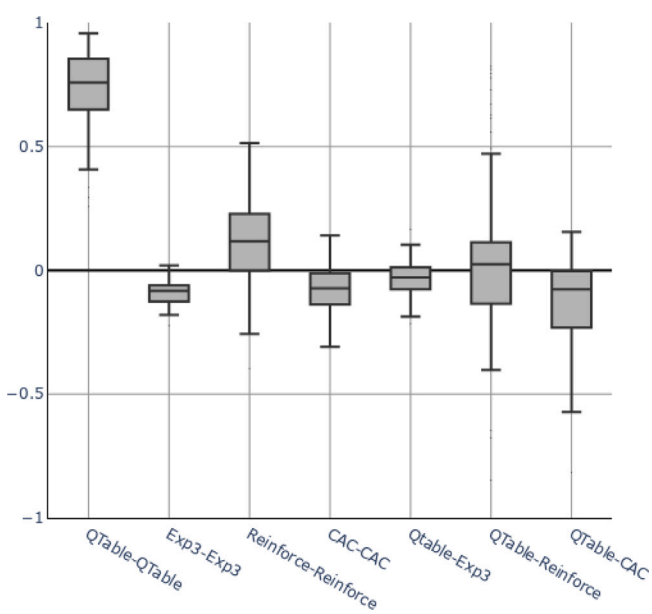

Fig. 7. Cooperation rate after learning for various algorithmic pairwise interactions. Ranges show the 10-90 percentiles of the distribution.

based on policy optimization. However, the conjecture is not proved theoretically and needs to be at least numerically tested with other algorithms in various competing environments.

It should be noted that the present section reports on the outcomes of numerous duels between pairs of algorithms, where we keep the hyperparameters of each algorithm constant throughout and across the duels. We acknowledge, however, that algorithmic performance is closely tied to the choice of hyperparameters, a choice that relies on the expertise of the data scientist. Given the unfeasibility of testing all hyperparameter combinations, we opted for a simple expositional approach: for each algorithm, we selected hyperparameters that consistently lead to the highest payoffs when competing against symmetric

Table 1
Normal-form representation of the supergame when managers can choose Q-learning or CAC (bold characters show average limit payoffs, standard font shows the limit standard deviation).

		Manager 2	
		Q-learning	CAC
Manager 1	Q-learning	**(12.13, 12.13)** (0.29, 0.29)	**(10.41, 11.42)** (0.50, 0.24)
	CAC	**(11.42, 10.41)** (0.24, 0.50)	**(11.00, 11.00)** (0.38, 0.38)

counterparts. Implicit in this process is the assumption that the hyperparameters optimizing performance against a peer also yield the best performance against other opponents. In a second step, we conducted a set of robustness checks, varying some hyperparameters such as the learning rate, the discount factor, the discretization of the action space, or the network size (CAC). These checks revealed that the results we report remained qualitatively unchanged for reasonable values of these hyperparameters. In the next section, we analyze in more detail the interaction between Q-learning and the other algorithms.

4.4. The choice of a seemingly-collusive algorithm may not be individually rational

The simulations ran so far naturally call for the analysis of a "supergame" whereby algorithmic implementation is a strategic choice of the managers. In a first stage, two managers, denoted by Manager 1 and Manager 2, choose simultaneously the algorithm they will implement. In the second stage, the algorithms get to interact through their environment and the managers receive the profits they generate after learning. We assume that each player may choose one of two types of algorithms: either Q-learning with $\epsilon_f = 0.1\%$ (the exploration rate that generates the highest payoffs with Q-learning according to Fig. 5) or CAC. (In a sensitivity analysis, we carried out a similar study by replacing CAC by Reinforce and Exp3 algorithms.) Both algorithms have base discount rates $\delta = 0.95$. For ease of exposition, we disregard the stream of payoffs during the learning process and focus on final payoffs only (those accrued in "play" mode, after the learning phase is complete). This renders account of a situation when the managers' discount factor is close to 1 or the learning phase is short relative to the exploitation phase, such that the profits during the learning phase can be neglected. Another justification could be that the learning phase is ran offline, within a synthetic and costless environment, though this assumes firms have access to the data-generating process. Each manager strives to optimize its profit.

In Fig. 7, one can already observe that the outcome of Q-learning vs CAC competition is close to the stage-game competition as the cooperation rate in the operational phase is on average close to 0 (albeit with a high variance across experiments). Table 1 shows the (limit) payoff matrix of the supergame (bold characters).

From Table 1, we draw two observations: first, the more sophisticated CAC algorithm consistently outperforms Q-learning when they play against each other. This means that choosing Q-learning is not a dominant strategy of the supergame. Second, the supergame does not have a unique Nash equilibrium where both managers choose the seemingly-collusive Q-learning algorithm. Both agents choosing the more sophisticated, and therefore the more competitive CAC algorithm, is also a Nash equilibrium of the supergame. In a sensitivity analysis, we have performed the same calculations replacing CAC by the Reinforce algorithm and found qualitatively similar results.

The case when managers can choose between Q-learning and Exp3, which we report in Table 2, is also insightful. In this case, both Q-learning/Q-learning and Exp3/Exp3 are Nash equilibria of the supergame and managers are almost indifferent between choosing Q-learning or Exp3, conditional on the other manager choosing

Table 2
Normal-form representation of the supergame when managers can choose Q-learning or Exp3 (bold characters show average limit payoffs, standard font shows the limit standard deviation).

		Manager 2	
		Q-learning	Exp3
Manager 1	Q-learning	**(12.13, 12.13)** (0.29, 0.29)	**(10.06, 12.08)** (0.14, 0.31)
	Exp3	**(12.08, 10.06)** (0.31, 0.14)	**(10.99, 10.99)** (0.03, 0.03)

Q-learning. While Q-learning/Q-learning is a payoff dominant joint strategy, it turns out that Exp3/Exp3 is risk-dominant. This suggests that the latter equilibrium may be quite plausible in real-world scenarios, as shown in previous studies (Schmidt, Shupp, Walker, and Ostrom (2003) and Maruta (1997)).

We acknowledge that, by essence, these simulations lack generality and our formulation of the supergame makes strong structural assumptions. For example, we assume that managers form accurate beliefs about payoffs and have to choose their algorithms simultaneously. When allowing for sequential choice of algorithms, both managers choosing Q-learning is the only Subgame-Perfect Nash Equilibrium of the games of Tables 1 and 2. Also, these payoff matrices focus on terminal profits and entirely neglect the cost of the learning phase itself. When considering the cost of the learning phase with a discount factor of 1 for example, CAC actually becomes a dominant strategy, owing to the very fast convergence in actions, compared to Q-learning (see e.g., Fig. 6). Nonetheless, this exercise highlights the importance of complementing the analysis of supracompetitive pricing by algorithms with a broader examination of the factors influencing algorithm selection. Overall, these findings provide empirical support for our Conjecture (3).

5. Policy recommendations

This paper illustrates theoretically and shows empirically that seeming collusion may be caused by excessively parsimonious exploration. Further, more sophisticated algorithms may outperform seemingly-collusive ones. Therefore, setting aside possible heterogeneity in implementation and learning costs, a manager's deliberate choice of a seemingly-collusive algorithm may not be individually rational. Such a choice does, however, allow to maximize joint profits: sophisticated algorithms compete more effectively against one another, and therefore would be avoided by cooperating managers. Therefore, the choice of an unsophisticated algorithm or of a low-exploration policy may in and of itself be a sign of (traditional) collusion, whereby managers would agree to use inefficient but seemingly-collusive algorithms so as to increase joint profits beyond the competitive levels. In light of these results, we recommend that competition authorities pay particular attention to collusion facilitated by algorithms, when the algorithmic exploration policy and algorithmic sophistication are low relative to the complexity of their environment. Most importantly, because we find that both managers choosing less sophisticated algorithms could also be a Nash equilibrium of the game of algorithmic selection, we also recommend regulators to develop adequate incentives to steer managers towards choosing more sophisticated algorithms to foster competition. This, of course, raises significant difficulties related to measuring algorithmic sophistication and problem complexity. These notions are indeed often described in rather qualitative terms and, in the context of algorithmic competition, might require resorting to a game theoretical approach as highlighted in Csató (2021). Even in the absence of a rigorous and universally-recognized measure of these parameters (as of yet), we believe that both these elements must be duly taken into account when authorities consider running an investigation of an algorithm.

As a consequence, this paper contributes to the debate on the necessity of adjusting existing antitrust regulation to algorithms. Faced with the complex task of demonstrating collusion, antitrust regulations and sanctions traditionally rest on formal evidence of a communication with competitors (Kaplow, 2013). Whether algorithms are actually able to make "conscious commitments" or "meeting of minds" is a thorny, rather ontological issue that may never be entirely resolved. As long as the issue remains unresolved, however, the possibility of algorithmic decision-making may offer a *carte blanche* to seeming collusion and any other socially undesirable actions. In this paper, we note that the choice of a specific algorithm remains a human decision, no different in nature from the choice of any other standard strategic variable (price, quantity...). In our setting, we show that the simultaneous and recurring choice of seemingly-collusive algorithms may not be individually rational, absent collusive motives. The availability of increasingly sophisticated algorithms may stimulate competition.

6. Conclusion

The fundamental drivers of algorithmic seeming collusion are still debated in the law and economics literatures. This paper investigates the extent to which the dynamics of exploration during learning may cause such an outcome. Through our analysis of competition between stylized Q-learners in a basic social dilemma framework, we exhibited the existence of a mirage effect: when algorithms find in the first exploratory iterations of learning that cooperation outperforms competition, and if furthermore the exploration rate decreases too rapidly, they might never learn to compete. We validate this result numerically through simulations. When analyzing the competition between various classes of algorithms, we find that increased sophistication may lead to more competition. As such, the choice of algorithmic non-sophistication could serve as an indicator of collusion among managers. This provides a new means for regulators to detect agents attempting to collude *via* the intermediation of algorithms.

This paper investigates only one potential driver of algorithmic (seeming) collusion but other drivers remain to be unveiled. In particular, we do not claim that algorithms may never learn genuine punishment strategies to sustain collusion in the long-term, nor that the absence of a "mirage effect" will always mitigate collusion. Furthermore, our research is limited to some classes of algorithms and other cases would need to be analyzed to generalize our results.

CRediT authorship contribution statement

Ibrahim Abada: Writing – review & editing, Writing – original draft, Validation, Supervision, Software, Resources, Project administration, Methodology, Formal analysis, Conceptualization. **Xavier Lambin:** Writing – review & editing, Writing – original draft, Validation, Supervision, Software, Project administration, Methodology, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Nikolay Tchakarov:** Validation, Supervision, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Acknowledgments

This project is supported by ADEME's funding program "Programme d'investissements d'avenir" (PIA). We warmly thank Arnoud V. den Boer, Janusz Maylahn, Joaquin Rodriguez, Isabella Seeber, Clara Jean, Charis Li, and seminar participants at HEC Paris, ESSEC Business School, Nanterre Université, as well as the Editor and three anonymous referees for their excellent comments and feedback.

Appendix A. Proofs and mathematical developments

To ease the notation, we introduce the following sequences to fully define the Q-matrix of Algorithm 1:

$$\forall n \in \mathbb{N},\ u_n = \left\langle \Pi_n^1(C) \right\rangle \tag{34}$$

$$\forall n \in \mathbb{N},\ v_n = \left\langle \Pi_n^1(D) \right\rangle. \tag{35}$$

and the following parameters:

$$a_1 = p_1\lambda_1 + (1-p_1) + (1-\lambda_1)\delta_1 p_1, \qquad a_1 \in (0,1) \tag{36}$$

$$b_1 = (1-\lambda_1)p_1\left(p_2\alpha + (1-p_2)\sigma\right), \qquad b_1 > 0 \tag{37}$$

$$a_2 = (1-p_1)\lambda_1 + p_1, \qquad a_2 \in (0,1) \tag{38}$$

$$c_2 = \delta_1(1-\lambda_1)(1-p_1), \qquad c_2 \in (0,1) \tag{39}$$

$$b_2 = (1-p_1)(1-\lambda_1)(p_2\phi + (1-p_2)\beta) \qquad b_2 > 0 \tag{40}$$

$$\mu = \frac{p_2\alpha + (1-p_2)\sigma}{1-\delta_1} \tag{41}$$

$$\nu = \mu + p_2(\phi - \alpha) + (1-p_2)(\beta - \sigma). \tag{42}$$

On the right-hand side, we highlight a couple of useful properties of these variables, that are easy to verify. As such, one can write the following recursive relations, replicating Eqs. (6) and (7) for Algorithm 1 (similar expressions can be written for Algorithm 2):

$$\forall n \in \mathbb{N},\ u_{n+1} = a_1 u_n + (1-\lambda_1)\delta_1 p_1\Big(\mathrm{Max}\left(u_n, v_n\right) - u_n\Big) + b_1 \tag{43}$$

$$v_{n+1} = a_2 v_n + c_2 u_n + c_2\Big(\mathrm{Max}\left(u_n, v_n\right) - u_n\Big) + b_2. \tag{44}$$

With these simpler notations, this section reports on the technical demonstrations of the main theorems of the paper.

A.1. Proof of Theorem 2

Proof. The proof is split in two steps. The first step analyzes what happens during learning (when $n \le N$) and the second step what happens during operations (when $n \ge N$).

Step 1: during learning. In this step, we prove that there exist a threshold N_1 for the first algorithms (respectively N_2 for the second) such that if $N \le N_1$ (respectively $N \le N_2$), then $\forall n \le N$, $\left\langle \Pi_n^1(D) \right\rangle \le \left\langle \Pi_n^1(C) \right\rangle$ ($\left\langle \Pi_n^2(D) \right\rangle \le \left\langle \Pi_n^2(C) \right\rangle$ respectively). We detail the proof for the first algorithm and observe that its line of reasoning applies directly to the second. To that end, we use the simplified notation of the preamble of Appendix A.

Recall that we assume that $u_0 > v_0$ and that $\xi\lambda_1 \ge \xi - (u_0 - v_0)$ (Assumption H1) with:

$$\xi = \left(p_1 + \delta_1 - 2\delta_1 p_1\right)u_0 - (1-p_1)v_0 + (1-p_1)\left(p_2\phi + (1-p_2)\beta\right) - p_1\left(p_2\alpha + (1-p_2)\sigma\right) \tag{45}$$

It can be demonstrated that condition $\xi\lambda_1 \ge \xi - (u_0 - v_0)$ is equivalent to

$$u_1 \ge v_1. \tag{46}$$

To prove the claim of the theorem, we proceed via weak induction. The result is true for $n = 0$ by assumption. If it is true for $k = 0, 1, \dots, n$, let us find the conditions that will also make it true for $k = n+1$. Because $\forall k \le n$, $v_k \le u_k$, one can then replace $\mathrm{Max}(u_k, v_k) - u_k$ by 0 in Eqs. (43) and (44) to get the following:

$$\forall k \le n,\ u_{k+1} = a_1 u_k + b_1 \tag{47}$$

$$v_{k+1} = a_2 v_k + c_2 u_k + b_2 \tag{48}$$

which defines two linear recursive expressions of the first order for u_k and v_k. Such a system can be solved using a matrix representation. The solution of relations (47) and (48) is then the following:

- When $a_1 \ne a_2$

$$\forall k \le n,\ u_k = \mu + a_1^k\left(u_0 - \mu\right) \tag{49}$$

$$v_k = \nu + \rho\left(a_1^k - a_2^k\right)\left(u_0 - \mu\right) + a_2^k(v_0 - \nu) \tag{50}$$

with

$$\rho = \frac{c_2}{a_1 - a_2} = \frac{\delta_1(1-p_1)}{1 - 2p_1 + \delta_1 p_1} \tag{51}$$

- and when $a_1 = a_2$

$$\forall k \le n,\ u_k = \mu + a_1^k\left(u_0 - \mu\right) \tag{52}$$

$$v_k = \nu + kc_2 a_1^{k-1}\left(u_0 - \mu\right) + a_1^k(v_0 - \nu). \tag{53}$$

Therefore:

$$\forall k \le n,\ u_k - v_k = a_1^k\Big[(u_0-\mu)(1-\rho)\Big] + a_2^k\Big[\rho(u_0-\mu) - (v_0-\nu)\Big] + (\mu - \nu) \text{ if } a_1 \ne a_2 \tag{54}$$

$$\forall k \le n,\ u_k - v_k = a_1^k\Big[(u_0-\mu) - (v_0-\nu)\Big] - kc_2 a_1^{k-1}\left(u_0 - \mu\right) + (\mu - \nu) \text{ if } a_1 = a_2. \tag{55}$$

We first focus on the case where $a_1 \ne a_2$. Let us study $u_k - v_k$ as a function of $k \in \mathbb{R}^+$ parametrized by $\rho \in \mathbb{R}$. We then define the real valued function $f_\rho : \mathbb{R}^+ \longrightarrow \mathbb{R}$ as follows:

$$\forall x \ge 0,\ f_\rho(x) = a_1^x\Big[(u_0-\mu)(1-\rho)\Big] + a_2^x\Big[\rho(u_0-\mu) - (v_0-\nu)\Big] + (\mu - \nu) \tag{56}$$

so that:

$$\forall k \le n,\ u_k - v_k = f_\rho(k). \tag{57}$$

We now analyze the variations of that function to look for the range where it is non-negative. We have $f_\rho(0) = u_0 - v_0 > 0$ and $\lim_{x \longrightarrow +\infty} = \mu - \nu < 0$. Function f_ρ is differentiable over $\mathbb{R}^+$ and its derivative is:

$$\forall x \ge 0,\ \frac{\partial f_\rho}{\partial x}(x) = \ln(a_1)a_1^x\Big[(u_0-\mu)(1-\rho)\Big] + \ln(a_2)a_2^x\Big[\rho(u_0-\mu) - (v_0-\nu)\Big]. \tag{58}$$

Three cases have to be considered:

- If $\rho \le 0$. This implies in particular that $a_1 < a_2$ because of Eq. (51). One can then easily show that:

$$\frac{\partial f_\rho}{\partial x}(x) \le 0 \Longleftrightarrow \left(\frac{a_1}{a_2}\right)^x \le \left(\frac{(v_0-\nu) - \rho(u_0-\mu)}{(1-\rho)(u_0-\mu)}\right)\frac{\ln(a_2)}{\ln(a_1)} \tag{59}$$

and we denote by τ_ρ the expression:

$$\tau_\rho = \left(\frac{(v_0-\nu) - \rho(u_0-\mu)}{(1-\rho)(u_0-\mu)}\right)\frac{\ln(a_2)}{\ln(a_1)} \tag{60}$$

so that

$$\frac{\partial f_\rho}{\partial x}(x) \le 0 \Longleftrightarrow \left(\frac{a_1}{a_2}\right)^x \le \tau_\rho \tag{61}$$

One can check that because $v_0 < u_0 < \mu < \nu$ (Assumption H1) and because $\rho \le 0$, one has:

$$\tau_\rho \ge 0 \tag{62}$$

and therefore (because $a_1 < a_2$):

$$\frac{\partial f_\rho}{\partial x}(x) \le 0 \Longleftrightarrow x \ge \frac{\ln(\tau_\rho)}{\ln(a_1) - \ln(a_2)}. \tag{63}$$

As a consequence, f_ρ starts at a positive value $(u_0 - v_0)$, increases then decreases before converging to a negative value $\mu - \nu$. Therefore, because $u_1 - v_1 > 0$, there exists $N_1 > 0$ such that

$$\forall k \in \mathbb{N},\ k \le N_1 \Longleftrightarrow f_\rho(k) \ge 0. \tag{64}$$

- If $\rho \geq 1$. In particular, this implies that $a_1 > a_2$. It can then be shown that:
$$\frac{\partial f_\rho}{\partial x}(x) \leq 0 \Longleftrightarrow \left(\frac{a_1}{a_2}\right)^x \geq \tau_\rho. \tag{65}$$
When $(v_0 - v) - \rho(u_0 - \mu) \leq 0 \Longleftrightarrow 1 \leq \rho \leq \frac{v_0 - v}{u_0 - \mu}$, then $\tau_\rho \leq 0$ and relation (65) always holds, which means that f_ρ is a decreasing function over $\mathbb{R}^+$. On the contrary, when $(v_0 - v) - \rho(u_0 - \mu) > 0 \Longleftrightarrow \rho \geq \frac{v_0 - v}{u_0 - \mu} \geq 1$, then $\tau_\rho \geq 0$ and relation (65) becomes equivalent to relation (63) (since $a_1 > a_2$). In both cases, following the previous line of reasoning, this implies the existence of $N_1 > 0$ such that
$$\forall k \in \mathbb{N},\ k \leq N_1 \Longleftrightarrow f_\rho(k) \geq 0. \tag{66}$$
- If $\rho \in (0, 1)$. In particular, this implies that $a_1 > a_2$ and one can show that:
$$\frac{\partial f_\rho}{\partial x}(x) \leq 0 \Longleftrightarrow \left(\frac{a_1}{a_2}\right)^x \leq \tau_\rho \tag{67}$$
and that in that case, we will have $\tau_\rho \geq 0$. Therefore:
$$\frac{\partial f_\rho}{\partial x}(x) \leq 0 \Longleftrightarrow x \leq \frac{\ln(\tau_\rho)}{\ln(a_1) - \ln(a_2)}. \tag{68}$$
Then, f_ρ starts at a positive value $(u_0 - v_0)$, decreases below $\mu - v$ then increases before converging to the negative value $\mu - v$. Therefore, here again, because $u_1 - v_1 \geq 0$, there exists $N_1 > 0$ such that
$$\forall k \in \mathbb{N},\ k \leq N_1 \Longleftrightarrow f_\rho(k) \geq 0. \tag{69}$$

To wrap-up the case where $a_1 \neq a_2$, as long as $n + 1 \leq N_1$, one can write:
$$u_{n+1} - v_{n+1} = f_\rho(n + 1) \geq 0 \tag{70}$$
and the induction carries over to $k = n + 1$.

Finally, when $a_1 = a_2$, a similar investigation of the right hand side term of Eq. (55) will allow to conclude to the existence of $N_1 > 0$ such that $u_k - v_k \geq 0$ whenever $k \leq N_1$, with
$$N_1 \geq \frac{a_1 \ln(a_1)\Big[(u_0 - \mu) - (v_0 - v)\Big] - c_2(u_0 - \mu)}{c_2 \ln(a_1)(u_0 - \mu)}$$
and the induction will carry over to the next Iteration $k = n+1$ as long as $n + 1 \leq N_1$. Relation (46) shows that $N_1 \geq 1$, which achieves Step 1 of the proof of the theorem.

Step 2: during operations. Building on the result of Step 1, we now analyze what happens during the operations phase of the algorithms if the thresholds N_1 and N_2 are not reached during learning (recall that in the operations mode, the Q-matrices are still updated). We define the new threshold N_{min} as follows:
$$N_{min} := \text{Min}\left(N_1, N_2\right) \tag{71}$$
with N_1 being the threshold below which function f_ρ is positive. By continuity, this threshold N_1 is a zero of function f_ρ defined in Eq. (56) which implies that (we only treat the general case where $a_1 \neq a_2$ but the methodology can be generalized following the same line of reasoning. The threshold N_2 of the second algorithm can be obtained in a similar manner):
$$a_1^{N_1}\Big[(u_0 - \mu)(1 - \rho)\Big] + a_2^{N_1}\Big[\rho(u_0 - \mu) - (v_0 - v)\Big] = v - \mu. \tag{72}$$
Let us prove that If $N \leq N_{min}$, then
$$\forall n \in \mathbb{N},\ \left\langle \Pi_n^1(D)\right\rangle \leq \left\langle \Pi_n^1(C)\right\rangle \text{ and } \left\langle \Pi_n^2(D)\right\rangle \leq \left\langle \Pi_n^2(C)\right\rangle. \tag{73}$$
We focus on the first algorithm (the demonstration can naturally be generalized to the second one). The proposition is true for $n \leq N$ by Step 1. We resort to weak induction to prove it for $n \geq N$. Let us assume that the proposition is true for $N,\ N + 1, \dots, N + k$ and let us prove it for $n = N + k + 1$. Then:
$$\forall l \in \{0, 1, \dots, k\}, \quad u_{N+l} \geq v_{N+l}. \tag{74}$$

After Iteration N (i.e. during the operational phase), algorithms do not explore anymore. As a consequence, at Iterations $N,\ N+1, \dots, N+k$, the algorithms consider that the profit accrued from cooperation is higher than that of competition on average, meaning that the optimal action is always to cooperate. By Eqs. (43) and (44), one has (because agent's best action is to cooperate, they set $p_1 = p_2 = 1$ during the operational phase):
$$\forall l \in \{N,\ N + 1, \dots, N + k\},\ u_{l+1} = \left(\lambda_1 + (1 - \lambda_1)\delta_1\right) u_l + (1 - \lambda_1)\alpha \tag{75}$$
$$v_{l+1} = v_l. \tag{76}$$

Using expression (75), one can write that $\forall l \in \{N, \dots, N + k\}$:
$$u_{l+1} - u_l = (1 - \lambda_1)(\delta_1 - 1)u_l + (1 - \lambda_1)\alpha = (1 - \lambda_1)(\alpha - (1 - \delta)u_l). \tag{77}$$

To finish the proof, we need the following intermediary result:

Lemma 1. *Under Assumption H1 and if $N \leq N_{min} = Min(N_1, N_2)$, then:*
$$\forall l \in \{1, 2, \dots, N + k + 1\},\ u_l \leq \frac{\alpha}{1 - \delta_1}. \tag{78}$$

Proof. To prove Lemma 1, we use the expression of u_l given in (49) for $l \leq N$ (during learning) and the recursive form (43) for $l \in \{N + 1, \dots, N + k\}$ during operations. Therefore, because $u_0 < \mu$, $u_l \leq \mu$ for $l \leq N$. The expression of μ is given in (41). Recalling that $\alpha > \sigma$, we observe that:
$$\mu \leq \frac{\alpha}{1 - \delta_1} \tag{79}$$
which proves the intermediary result for $l \leq N$. For l ranging between $N + 1$ and $N + k + 1$, we prove the result by weak induction starting from $l = N$ (which initiates the induction as just proved). Assume then that the result holds for $l \in \{N, \dots, N + k\}$. After Iteration N (i.e. during the operational phase), algorithms do not explore anymore, play only their optimal action, and update their Q-tables accordingly. As a consequence, after Iteration N, the algorithms consider that the profit accrued from cooperation is on average higher than that of competition, meaning that the optimal action is always to cooperate. One can then resort to Eq. (47) to model the dynamics of u_n, which gives (because agent's best action is to cooperate, they set $p_1 = p_2 = 1$ during the operational phase):
$$u_{l+1} \leq \left(\lambda_1 + (1 - \lambda_1)\delta_1\right)\frac{\alpha}{1 - \delta_1} + (1 - \lambda_1)\alpha = \frac{\alpha}{1 - \delta_1} \tag{80}$$
which concludes the proof of the lemma. □

We can now finish the proof of Step 2. From (77) and Lemma 1, we get that: $\forall l \in \{N, \dots, N + k\}$ $u_{l+1} - u_l \geq 0$. Therefore, one will get: $u_{N+k+1} \geq u_{N+k} \geq \cdots \geq u_N \geq v_N$ and because the value of v_n is not updated after learning (when $n \geq N$, recall that only cooperation is played after learning), one concludes that:
$$u_{N+k+1} \geq v_{N+k+1} \tag{81}$$
which proves the result for the first algorithm. An identical line of reasoning can be carried out for the second algorithm to finish the proof. □

A.2. Proof of Theorem 3

Proof. Variations with respect to p_2.

Recall that $N_{min}(p_1, p_2) = \text{Min}\left(N_1(p_1, p_2), N_2(p_1, p_2)\right)$ as defined in the proof of Theorem 2. To demonstrate that $N_{min}(p_1, p_2)$ is an increasing function of p_2, it suffices to prove it for both $N_1(p_1, p_2)$ and $N_2(p_1, p_2)$. We display the proof for $N_1(p_1, p_2)$ and observe that its line of reasoning naturally applies to the second algorithm.

The threshold N_1 is the zero of function f_ρ defined in Eq. (56) which implies that (in this proof, we only treat the general case where $a_1 \neq a_2$

but the methodology can be generalized following the same line of reasoning):

$$a_1^{N_1}\left[(u_0-\mu)(1-\rho)\right]+a_2^{N_1}\left[\rho(u_0-\mu)-(v_0-\nu)\right]=\nu-\mu. \tag{82}$$

We observe that a_1, a_2, and ρ do not depend on p_2 and that all the terms (except N_1 that we will discuss soon) involved in Eq. (82) are differentiable with respect to p_2. Using the implicit functions theorem (that we will justify later on), it can be shown that N_1 is also differentiable with respect to p_2. As a consequence, one can write (by differentiating (82) with respect to p_2) that:

$$-(1-\rho)\frac{\partial\mu}{\partial p_2}a_1^{N_1}+\left[-\rho\frac{\partial\mu}{\partial p_2}+\frac{\partial\nu}{\partial p_2}\right]a_2^{N_1}+\kappa\frac{\partial N_1}{\partial p_2}=\frac{\partial\nu}{\partial p_2}-\frac{\partial\mu}{\partial p_2} \tag{83}$$

with

$$\kappa(N_1)=\left[(u_0-\mu)(1-\rho)\ln(a_1)a_1^{N_1}+\left[\rho(u_0-\mu)-(v_0-\nu)\right]\ln(a_2)a_2^{N_1}\right]. \tag{84}$$

The implicit functions theorem applies because the term $\kappa(N_1)$ is negative as it is actually equal to the derivative of function f_ρ with respect to x expressed at N_1, as shown in relation (58). Because N_1 is the zero of f_ρ, it is never an optimum of that function (see the analysis of the variations of function f_ρ provided in the Proof of Theorem 2). In particular, $\kappa(N_1)\neq 0$, which allows us to resort to the implicit functions theorem. Using the expressions of μ and ν reported in relations (8) and (9), one can write (to ease the notation, we will write κ instead of $\kappa(N_1)$ in the rest of our developments, when no confusion is possible):

$$\begin{aligned}&-(1-\rho)\frac{\alpha-\sigma}{1-\delta_1}a_1^{N_1}+\left[(-\rho+1)\frac{\alpha-\sigma}{1-\delta_1}+(\phi-\alpha-\beta+\sigma)\right]a_2^{N_1}+\kappa\frac{\partial N_1}{\partial p_2}\\&\quad=(\phi-\alpha-\beta+\sigma)\end{aligned} \tag{85}$$

or

$$(1-\rho)\frac{\alpha-\sigma}{1-\delta_1}\left(a_2^{N_1}-a_1^{N_1}\right)+\kappa\frac{\partial N_1}{\partial p_2}=(\phi-\alpha-\beta+\sigma)\left(1-a_2^{N_1}\right) \tag{86}$$

or

$$\kappa\frac{\partial N_1}{\partial p_2}=(\phi-\alpha-\beta+\sigma)\left(1-a_2^{N_1}\right)-(1-\rho)\frac{\alpha-\sigma}{1-\delta_1}\left(a_2^{N_1}-a_1^{N_1}\right). \tag{87}$$

Because $\kappa<0$, it suffices then to show that the right-hand-side of Eq. (87) is non-positive to complete the proof. This is equivalent to:

$$\begin{aligned}(\phi-\alpha-\beta+\sigma)&\leq(1-\rho)\frac{\alpha-\sigma}{1-\delta_1}\left[\frac{a_2^{N_1}-a_1^{N_1}}{1-a_2^{N_1}}\right]\\&=(c_2+a_2-a_1)\frac{\alpha-\sigma}{(1-\delta_1)(1-a_2^{N_1})}\left[\frac{a_2^{N_1}-a_1^{N_1}}{a_2-a_1}\right]\end{aligned} \tag{88}$$

where the equality derives from the definition of ρ (Eq. (51)). Observing that $\alpha>\sigma$ and $c_2+a_2-a_1=(2p_1-1)(1-\lambda_1)(1-\delta_1)$, we have that the condition is satisfied for all $p_1\geq\frac{1}{2}$. This proves the theorem.

Variations with respect to p_1.

Here again, because $N_{min}(p_1,p_2)=\text{Min}\left(N_1(p_1,p_2),N_2(p_1,p_2)\right)$, it suffices to prove the claim for $N_1(p_1,p_2)$ and $N_2(p_1,p_2)$. We display the proof for $N_1(p_1,p_2)$ and observe that its line of reasoning naturally applies to the second algorithm.

The threshold N_1 is the zero of function f_ρ (here again, we assume $a_1\neq a_2$):

$$a_1^{N_1}\left[(u_0-\mu)(1-\rho)\right]+a_2^{N_1}\left[\rho(u_0-\mu)-(v_0-\nu)\right]=\nu-\mu \tag{89}$$

We observe that μ and ν do not depend on p_1 and that all the terms (except N_1 that we discuss later on) involved in Eq. (89) are differentiable with respect to p_1. The derivatives are as follows:

$$\frac{\partial a_1}{\partial p_1}=(1-\lambda_1)(\delta_1-1) \tag{90}$$

$$\frac{\partial a_2}{\partial p_1}=(1-\lambda_1) \tag{91}$$

$$\frac{\partial\rho}{\partial p_1}=\frac{\delta_1(1-\delta_1)}{\left(1-(2-\delta_1)p_1\right)^2}. \tag{92}$$

Furthermore, using the implicit functions theorem, it can be shown that N_1 is also differentiable with respect to p_1 (a justification of this will be provided soon). By differentiating (89) with respect to p_1, one gets:

$$\begin{aligned}&a_1^{N_1}\left[\frac{\partial N_1}{\partial p_1}\ln(a_1)+\frac{N_1}{a_1}(1-\lambda_1)(\delta_1-1)\right](u_0-\mu)(1-\rho)\\&\quad+a_1^{N_1}(u_0-\mu)\frac{\delta_1(\delta_1-1)}{\left(1-(2-\delta_1)p_1\right)^2}\\&\quad+a_2^{N_1}\left[\frac{\partial N_1}{\partial p_1}\ln(a_2)+\frac{N_1}{a_2}(1-\lambda_1)\right]\left[\rho(u_0-\mu)-(v_0-\nu)\right]\\&\quad+a_2^{N_1}(u_0-\mu)\frac{\delta_1(1-\delta_1)}{\left(1-(2-\delta_1)p_1\right)^2}=0.\end{aligned} \tag{93}$$

By rearranging the different terms, we obtain the following:

$$\frac{\partial N_1}{\partial p_1}\frac{\partial f_\rho}{\partial x}(N_1)+(1-\lambda_1)N_1\zeta+\frac{\delta_1(1-\delta_1)}{\left(1-(2-\delta_1)p_1\right)^2}(u_0-\mu)\left(a_2^{N_1}-a_1^{N_1}\right)=0 \tag{94}$$

where $\frac{\partial f_\rho}{\partial x}$ is the derivative of function f_ρ that is expressed in Eq. (58) and ζ is equal to:

$$\zeta=\left[a_1^{N_1-1}(\delta_1-1)(u_0-\mu)(1-\rho)+a_2^{N_1-1}\left[\rho(u_0-\mu)-(v_0-\nu)\right]\right]. \tag{95}$$

The derivative of function f_ρ is always negative at its zero N_1 (see the analysis of the variations of function f_ρ given in the Proof of Theorem 3). This justifies the recourse to the implicit functions theorem to prove the differentiability of function N_1 with respect to p_1. Besides, using Eq. (89), we can rewrite the expression of ζ as follows:

$$\zeta=\left[\frac{(\nu-\mu)}{a_2}-(u_0-\mu)(1-\rho)a_1^{N_1-1}\left(\frac{a_1}{a_2}-(\delta_1-1)\right)\right]. \tag{96}$$

Plugging this last expression in (94), one gets:

$$\begin{aligned}&\frac{\partial N_1}{\partial p_1}\frac{\partial f_\rho}{\partial x}(N_1)+(1-\lambda_1)N_1\left[\frac{(\nu-\mu)}{a_2}-(u_0-\mu)(1-\rho)a_1^{N_1-1}\left(\frac{a_1}{a_2}-(\delta_1-1)\right)\right]\\&\quad+\frac{\delta_1(1-\delta_1)}{\left(1-(2-\delta_1)p_1\right)^2}(u_0-\mu)\left(a_2^{N_1}-a_1^{N_1}\right)=0\end{aligned} \tag{97}$$

or that (recall that $N_1>0$)

$$\begin{aligned}&-\frac{1}{N_1}\frac{\partial N_1}{\partial p_1}\frac{\partial f_\rho}{\partial x}(N_1)=(1-\lambda_1)\left[\frac{(\nu-\mu)}{a_2}-(u_0-\mu)(1-\rho)a_1^{N_1-1}\left(\frac{a_1}{a_2}-(\delta_1-1)\right)\right]\\&\quad+\frac{\delta_1(1-\delta_1)}{\left(1-(2-\delta_1)p_1\right)^2}(u_0-\mu)\left(a_2^{N_1}-a_1^{N_1}\right).\end{aligned} \tag{98}$$

We recall the expressions of parameters μ, ν, a_1, a_2, and ρ:

$$\mu=\frac{p_2\alpha+(1-p_2)\sigma}{1-\delta_1} \tag{99}$$

$$\nu=\mu+p_2(\phi-\alpha)+(1-p_2)(\beta-\sigma) \tag{100}$$

$$a_1=p_1\lambda_1+(1-p_1)+(1-\lambda_1)\delta_1p_1 \tag{101}$$

$$a_2=(1-p_1)\lambda_1+p_1 \tag{102}$$

$$1-\rho=\frac{(1-\delta_1)(1-2p_1)}{(1-2p_1+\delta_1p_1)}. \tag{103}$$

Therefore, when $\delta_1\longrightarrow 1$, one has:

$$\mu(1-\rho)\longrightarrow\mu'=\left(p_2\alpha+(1-p_2)\sigma\right)\frac{(1-2p_1)}{(1-p_1)} \tag{104}$$

$$a_1\longrightarrow 1 \tag{105}$$

$$a_2\longrightarrow(1-p_1)\lambda_1+p_1 \tag{106}$$

$$\rho\longrightarrow 1 \tag{107}$$

$$\nu-\mu\longrightarrow p_2(\phi-\alpha)+(1-p_2)(\beta-\sigma). \tag{108}$$

Variable μ' is negative because $p_1\geq\frac{1}{2}$. We now analyze the behavior of N_1 when $\delta_1\longrightarrow 1$. We then start from Eq. (89) and analyze its limit. The following lemma gives the main results.

Lemma 2. *Under Assumptions H1–H4, we have:*

- $\nu - \mu + \mu' > 0 \iff N_1$ *is finite and satisfies:*

$$a_2^{N_1} = \frac{\nu - \mu + \mu'}{(\nu - \mu + u_0 - v_0)} \in [0, 1] \tag{109}$$

- $\nu - \mu + \mu' \leq 0 \iff N_1$ *is infinite.*

Proof of Lemma 2.

Proof. We recall that N_1 is solution of:

$$a_1^{N_1}\left[(u_0 - \mu)(1 - \rho)\right] + a_2^{N_1}\left[\rho(u_0 - \mu) - (v_0 - \nu)\right] = \nu - \mu. \tag{110}$$

The following reasoning is conducted by considering a δ_1-depending sequence of N_1 and analyzing converging or diverging sub-sequences when $\delta_1 \longrightarrow 1$. Two cases have to be considered:

- If $\nu - \mu + \mu' > 0$. Assume then that N_1 is unbounded when $\delta_1 \longrightarrow 1$. Since $a_2 < 1$, any diverging sub-sequence has then to satisfy:

$$-\mu' a_1^N = \nu - \mu \tag{111}$$

or

$$a_1^N = -\frac{\nu - \mu}{\mu'} > 1 \tag{112}$$

which is impossible. This implies that N_1 is bounded when $\delta_1 \longrightarrow 1$. Since $a_1 \longrightarrow 1$, any converging sub-sequence has then a limit N that satisfies:

$$-\mu' + a_2^N\left(\nu - \mu + u_0 - v_0\right) = \nu - \mu \tag{113}$$

or

$$a_2^N = \frac{\nu - \mu + \mu'}{(\nu - \mu + u_0 - v_0)} \geq 0 \tag{114}$$

which implies that there is a unique accumulation point. Hence, at the limit $\delta_1 \longrightarrow 1$, N_1 satisfies:

$$a_2^{N_1} = \frac{\nu - \mu + \mu'}{(\nu - \mu + u_0 - v_0)} \in (0, 1). \tag{115}$$

- If $\nu - \mu + \mu' \leq 0$. A similar line of reasoning will reveal that N_1 has no finite accumulation point and has to be unbounded when $\delta_1 \longrightarrow 1$. Any diverging sub-sequence has then to satisfy:

$$-\mu' a_1^N = \nu - \mu \tag{116}$$

or

$$a_1^N = -\frac{\nu - \mu}{\mu'} \leq 1 \tag{117}$$

which implies that N_1 is infinite because $a_1 \longrightarrow 1$.

A summary of the previous analysis shows that at the limit ($\delta_1 \longrightarrow 1$), N_1 is either infinite when $\nu - \mu + \mu' \leq 0$ or otherwise satisfies:

$$a_2^{N_1} = \frac{\nu - \mu + \mu'}{(\nu - \mu + u_0 - v_0)}. \tag{118}$$

The reverse implications can be demonstrated in a similar way by analyzing the limits of all the terms of Eq. (110). □

We now resume the proof of Theorem 3. From Lemma 2, we have that N_1 is either infinite or is finite when $\nu - \mu + \mu' > 0$. In that case, at the limit $\delta_1 \longrightarrow 1$, relation (98) simplifies into:

$$-\frac{1}{N_1}\frac{\partial N_1}{\partial p_1}\frac{\partial f_\rho}{\partial x}(N_1) = (1 - \lambda_1)\left[\frac{(\nu - \mu + \mu')}{a_2}\right] - \frac{\mu'}{(1 - p_1)(1 - 2p_1)}\left(a_2^{N_1} - 1\right). \tag{119}$$

Considering the fact that $\mu' \leq 0$, $p_1 \geq \frac{1}{2}$ and that $a_2^{N_1} - 1 \leq 0$, one has

$$\frac{1}{N_1}\frac{\partial N_1}{\partial p_1}\frac{\partial f_\rho}{\partial x}(N_1) \leq 0 \tag{120}$$

or that

$$\frac{\partial N_1}{\partial p_1} \geq 0 \tag{121}$$

which proves the claim. □

A.3. A mathematical formulation of the mirage effect

This section fully formalizes the mirage effect and provides a sufficient condition on the decreasing rate of exploration that triggers it when the discount rates are close to 1. In order to do this, we need to consider the general case where the exploration policy decreases with the algorithms' Iterations n by explicitly making $\epsilon_i(n)$ and consequently $p_1(n)$ and $p_2(n)$ depend on n in the system of recursive Eqs. (6) and (7). We also analyze the extreme case where the duration of learning is infinite ($N \longrightarrow +\infty$) which makes it, in theory, more difficult for algorithms to cooperate (by Theorem 1). This general problem, which we denote by (P) will be approximated by a series of learning experiments defined for each Iteration $n \in \mathbb{N}$ by some Q-learning recursive equations, denoted by $(P)^n$. To ease the notation, we report these equations only for Algorithm 1 and do not use a superscript indicating to which algorithm notation refers for problems $(P)^n$ and problem (P). Our results will naturally carry over to the second algorithm. Problem $(P)^n$ is defined as follows (to ease the notation, we do not write the expectation operator $\langle . \rangle$ around profits): $\forall k \in \mathbb{N}$,

$$\begin{aligned}\Pi_{k+1}^n(C) &= a_1(n)\Pi_k^n(C) + (1 - \lambda_1)\delta_1 p_1(n) \\ &\quad \left(\text{Max}\left(\Pi_k^n(C), \Pi_k^n(D)\right) - \Pi_k^n(C)\right) + b_1(n)\end{aligned} \tag{122}$$

$$\begin{aligned}\Pi_{k+1}^n(D) &= a_2(n)\Pi_k^n(D) + c_2(n)\Pi_k^n(C) \\ &\quad + c_2(n)\left(\text{Max}\left(\Pi_k^n(C), \Pi_k^n(D)\right) - \Pi_k^n(C)\right) + b_2(n)\end{aligned} \tag{123}$$

with parameters $a_1(n)$, $a_2(n)$, $c_2(n)$, $b_1(n)$, and $b_2(n)$ defined by making p_i depend on n in Eqs. (36)–(40). The initial conditions of the problems $(P)^n$ are left unchanged ($\Pi_0^1(C)$ and $\Pi_0^1(D)$). In other words, for an Iteration n, problem $(P)^n$ models the expectation of a Q-learning experiment where the probabilities of choice of actions is kept equal to $p_i(n)$ during the whole learning process (and for which we have already derived some theoretical results). As in the proof of Theorem 2, we denote thresholds of problems $(P)^n$ by $N_1(n)$ and $N_2(n)$ as they also depend on n. Recall that $N_1(n)$ (respectively $N_2(n)$) is the threshold above which Algorithm 1 (respectively Algorithm 2) will learn that competition outperforms cooperation on average. We now conduct the analysis by considering the series of problems $(P)^n$ and we will then discuss how our results can be used in the analysis of problem (P). To further simplify the exposition, we will consider Iteration $n \in \mathbb{R}$ as a real-valued number with respect to which we will be able to differentiate some functions.

Definition 1. The mirage effect of imperfect learning occurs when algorithms cannot learn that competition can outperform competition in expectation, under the learning process of problems $(P)^n$. Put into equations, the mirage occurs when:

$$\forall n \in \mathbb{N},\ \forall i \in \{1, 2\},\ \frac{\partial N_i(n)}{\partial n} > 1 \tag{124}$$

and as consequence, we will have:

$$\forall n \in \mathbb{N},\ \forall i \in \{1, 2\},\quad N_i(n) > n. \tag{125}$$

More precisely, if the thresholds $N_1(n)$ and $N_2(n)$ (and consequently $N_{min}(n)$) increase faster than the iterations, which materializes when $\frac{\partial N_i(n)}{\partial n} > 1$, they will never be reached during learning as stated in Eq. (125). Because cooperation will be the dominant action (as perceived by the Q-matrices in expectation) when these thresholds are not reached (as per Theorem 2), algorithms might not learn to compete. We now focus on the first algorithm and assume for simplicity that (for the rest of this section) p_1 is kept constant with only $p_2(n)$ changing with

iteration. Furthermore, we classically assume that $p_2(0) = \frac{1}{2}$. We also assume that the second algorithm is in the mode where it considers that cooperation still outperforms competition and hence *increases* $p_2(n) = 1 - \epsilon_2(n)$ with iteration. To help the analysis, we define the following set:

$$H \equiv \left\{n \in \mathbb{N} \text{ such that } p_1 \leq e(n)\right\} \tag{126}$$

with the sequence $e(n)$ defined as follows:

$$\forall n \in \mathbb{N},\ e(n) \equiv \frac{p_2(n)\phi + (1-p_2(n))\beta}{p_2(n)(\phi+\alpha) + (1-p_2(n))(\beta+\sigma)}. \tag{127}$$

It can be shown that when $\phi\sigma \leq \alpha\beta$, $e(n)$ decreases with n. We denote by $p_{2,f}$ the limit of $p_2(n)$ when $n \longrightarrow +\infty$ (recall that $p_2(n)$ is an increasing and bounded sequence). In other words, $p_{2,f}$ is the final probability to play cooperation for the second algorithm. Translating this to sequence $e(n)$, we introduce the following notation:

$$e_f \equiv \frac{p_{2,f}\phi + (1-p_{2,f})\beta}{p_{2,f}(\phi+\alpha) + (1-p_{2,f})(\beta+\sigma)}. \tag{128}$$

As a consequence, when $p_1 > e(0) = \frac{\phi+\beta}{\phi+\beta+\alpha+\sigma}$ (and when $\phi\sigma \leq \alpha\beta$), set H is empty. When $e(0) \geq p_1 > e_f$, H is non-empty and bounded and we denote by n^* its maximum. Finally, when $p_1 \leq e_f$, H contains all integers. This leads us to introduce Assumption H5 (see Section 3.3) that ensures that the sequence $e(n)$ decreases. The following lemma focuses on problems $(P)^n$ to analyze the occurrence of a mirage.

Lemma 3. *Assume H1 for both algorithms and H2 to H5. There exists a positive constant (independent of p_2) $m > 0$ such that:*

1. *If $p_1 > e(0)$, then $\forall n \in \mathbb{N},\ N_1(n) = +\infty$.*
2. *If $e(0) \geq p_1 > e_f$ and if $\forall n \leq n^*,\ \frac{\partial p_2}{\partial n}(n) \geq m$, then $\forall n \leq n^*\ \frac{\partial N_1(n)}{\partial n} > 1$ and $\forall n \in \mathbb{N}, N_1(n) > n$.*

Lemma 3 directly implies the existence of a mirage (in the sense of Definition 1) and consequently $\forall n \in \mathbb{N}, \left\langle \Pi_n^n(C) - \Pi_n^n(D)\right\rangle \geq 0$. The link between thresholds $N_1(n)$ of problems $(P)^n$ and the initial problem (P) is given by the following result which is reformulated as Theorem 4 of Section 3.3 (and which we discuss in Appendix A.6 of the Appendix):

Theorem 5 (*A Reformulation of Theorem 4*). *Assume H1 for both algorithms and H2 to H5. If $\forall n \leq n^*,\ \frac{\partial p_2}{\partial n}(n) \geq m$ with m set as in Lemma 3 and if after learning, algorithms play their greedy action based on the expectation of their Q-matrices, then there exists a constant (independent of p_2) $q > 0$ such that if $\forall n \in \mathbb{N},\ \frac{\partial p_2}{\partial n}(n) \leq q$:*

$$p_1 > e_f = \frac{p_{2,f}\phi + (1-p_{2,f})\beta}{p_{2,f}(\phi+\alpha) + (1-p_{2,f})(\beta+\sigma)} \Longrightarrow \forall n \in \mathbb{N},\ \langle \Pi_n(C) - \Pi_n(D)\rangle \geq 0. \tag{129}$$

A.4. Proof of Lemma 3

Proof. We recall the expressions of parameters μ, ν, a_1, a_2, and ρ, some of which now depend on n (recall that p_2 depends on n):

$$\mu(n) = \frac{p_2(n)\alpha + (1-p_2(n))\sigma}{1-\delta_1} \tag{130}$$

$$\nu(n) = \mu(n) + p_2(n)(\phi-\alpha) + (1-p_2(n))(\beta-\sigma) \tag{131}$$

$$a_1 = p_1\lambda_1 + (1-p_1) + (1-\lambda_1)\delta_1 p_1 \tag{132}$$

$$a_2 = (1-p_1)\lambda_1 + p_1 \tag{133}$$

$$1-\rho = \frac{(1-\delta_1)(1-2p_1)}{(1-2p_1+\delta_1 p_1)}. \tag{134}$$

Therefore, when $\delta_1 \longrightarrow 1$, one has:

$$\mu(n)(1-\rho) \longrightarrow \mu'(n) = \left(p_2(n)\alpha + (1-p_2(n))\sigma\right)\frac{(1-2p_1)}{(1-p_1)} \tag{135}$$

$$a_1 \longrightarrow 1 \tag{136}$$

$$a_2 \longrightarrow (1-p_1)\lambda_1 + p_1 \tag{137}$$

$$\rho \longrightarrow 1 \tag{138}$$

$$\nu(n) - \mu(n) \longrightarrow p_2(n)(\phi-\alpha) + (1-p_2(n))(\beta-\sigma). \tag{139}$$

$\mu'(n)$ is non-positive because $p_1 \geq \frac{1}{2}$ (because $p_1 > e(f)$ by assumption, one will have $p_1 \geq \frac{1}{2}$ because $e(f) \geq \frac{1}{2}$). It can be shown that:

$$\nu(n) + \mu(n) + \mu'(n) \geq 0 \Longleftrightarrow p_1 \leq e(n) \tag{140}$$

with $e(n)$ the decreasing sequence defined in Eq. (127). As a consequence, when $p_1 > e(0)$, then $\forall n \in \mathbb{N},\ \nu(n) + \mu(n) + \mu'(n) < 0$ and using Lemma 2, this will imply that $\forall n \in \mathbb{N},\ N_1(n) = +\infty$. When $e(0) \geq p_1 > e_f$, we will have:

$$\forall n \leq n^*,\ p_1 \leq e(n) \text{ and } \forall n > n^*,\ p_1 > e(n) \tag{141}$$

meaning in particular that $\forall n > n^*,\ N_1(n) = +\infty$. We now analyze what happens when $n \leq n^*$. In that situation, Lemma 2 indicates that threshold N_1, that satisfies Eq. (82), is always positive and bounded when $\delta_1 \longrightarrow 1$. Let us then calculate the derivative of N_1 with respect to n. We first express the derivative of N_1 with respect to p_2 as in Eq. (87) (we hence do not explicitly write the dependence of p_2 on n for the time being):

$$\kappa(N_1)\frac{\partial N_1}{\partial p_2} = (\phi-\alpha-\beta+\sigma)\left(1-a_2^{N_1}\right) - (1-\rho)\frac{\alpha-\sigma}{1-\delta_1}\left(a_2^{N_1} - a_1^{N_1}\right) \tag{142}$$

with

$$\kappa(N_1) = \left[(u_0-\mu)(1-\rho)\ln(a_1)a_1^{N_1} + \left[\rho(u_0-\mu) - (v_0-\nu)\right]\ln(a_2)a_2^{N_1}\right]. \tag{143}$$

Lemma 2 gives:

$$a_2^{N_1} = \frac{\nu-\mu+\mu'}{(\nu-\mu+u_0-v_0)}. \tag{144}$$

Taking Eq. (143) to its limit, one gets:

$$\kappa(N_1) = (\nu-\mu+u_0-v_0)\ln(a_2)a_2^{N_1} = (\nu-\mu+\mu')\ln(a_2). \tag{145}$$

Taking Eq. (142) to its limit, one gets:

$$\begin{aligned}(\nu-\mu+\mu')\ln(a_2)\frac{\partial N_1}{\partial p_2} &= \left(1-a_2^{N_1}\right)\left[(\phi-\alpha-\beta+\sigma) - \frac{(2p_1-1)}{(1-p_1)}(\alpha-\sigma)\right] \\ &= \left(\frac{u_0-v_0-\mu'}{(\nu-\mu+u_0-v_0)}\right)\left[(\phi-\alpha-\beta+\sigma) - \frac{(2p_1-1)}{(1-p_1)}(\alpha-\sigma)\right]\end{aligned} \tag{146}$$

where the last equality is obtained via Eq. (144). If we denote by s the non-negative term (the non-negativity is obtained by assumption):

$$s = \frac{\left[(\phi-\alpha-\beta+\sigma) - \frac{(2p_1-1)}{(1-p_1)}(\alpha-\sigma)\right]}{\ln(a_2)} \tag{147}$$

and when we observe that s does not depend on p_2, we can rewrite the (at the limit) expression of $\frac{\partial N_1}{\partial p_2}$ as follows:

$$\frac{\partial N_1}{\partial p_2} = \left(\frac{u_0-v_0-\mu'}{(\nu-\mu+u_0-v_0)(\nu-\mu+\mu')}\right)s. \tag{148}$$

Because p_2 is now a function of Iteration n, one can use the chain rule and find the derivative of threshold N_1 with respect to n as:

$$\frac{\partial N_1}{\partial n}(n) = \left(\frac{u_0-v_0-\mu'(n)}{(\nu(n)-\mu(n)+u_0-v_0)(\nu(n)-\mu(n)+\mu'(n))}\right)s\frac{\partial p_2}{\partial n}(n). \tag{149}$$

Now, we know that:

$$\begin{aligned}\nu(n)-\mu(n)+u_0-v_0 &= u_0-v_0+p_2(n)(\phi-\alpha)+(1-p_2(n))(\beta-\sigma) \\ &\leq (u_0-v_0)+\phi-\alpha+\beta-\sigma.\end{aligned} \tag{150}$$

We also have (recall that $\mu'(n) \leq 0$):

$$\begin{aligned}\nu(n)-\mu(n)+\mu(n)' &= p_2(n)(\phi-\alpha)+(1-p_2)(n)(\beta-\sigma)+\mu' \\ &\leq \phi-\alpha+\beta-\sigma\end{aligned} \tag{151}$$

and

$$u_0 - v_0 - \mu'(n) \geq u_0 - v_0 > 0. \tag{152}$$

Plugging these inequalities in relation (149), we obtain:

$$\frac{\partial N_1}{\partial n}(n) \geq \left(\frac{u_0 - v_0}{(u_0 - v_0 + \phi - \alpha + \beta - \sigma)(\phi - \alpha + \beta - \sigma)} \right) s \frac{\partial p_2}{\partial n}(n). \tag{153}$$

We now denote by m the (independent from p_2) positive parameter:

$$m \equiv \frac{1}{s} \left(\frac{(u_0 - v_0 + \phi - \alpha + \beta - \sigma)(\phi - \alpha + \beta - \sigma)}{u_0 - v_0} \right) \tag{154}$$

We then have that if

$$\forall n \leq n^*, \frac{\partial p_2}{\partial n}(n) \geq m \tag{155}$$

then

$$\forall n \leq n^*, \frac{\partial N_1}{\partial n}(n) \geq 1 \tag{156}$$

which proves the claim. We may now immediately derive that:

$$\forall n \leq n^*, N_1(n) \geq n + N_1(n = 0) > n. \tag{157}$$

To wrap-up: under condition (155), when $p_1 > e(0)$, then $\forall n \in \mathbb{N}$, $N_1(n) = +\infty$. When $e(0) \geq p_1 > e_f$, we have: $\forall n \leq n^*, N_1(n) \geq n$ and $\forall n > n^*$, $N_1(n) = +\infty$: in expectation, algorithms get locked in the seemingly-collusive state. □

A.5. Proof of Theorem 4 (reformulated as Theorem 5)

Proof. We recall that only $p_2(n)$ depends on n and that p_1 is kept constant. We focus the analysis on Algorithm 1 and do not precise this in the subscripts of our notation anymore. To ease even further the notation, we will also resort to variables u_n, v_n, u_n^n, and v_n^n as follows (note that variables u_n and v_n now refer to the elements of the Q-matrix of the first algorithm for the general problem (P)):

$$\text{Problem } (P) \quad u_n = \langle \Pi_n(C) \rangle \qquad v_n = \langle \Pi_n(D) \rangle \tag{158}$$

$$\text{Problem } (P)^n \quad u_k^n = \left\langle \Pi_k^n(C) \right\rangle \qquad v_k^n = \left\langle \Pi_k^n(D) \right\rangle. \tag{159}$$

We also recall some notation of the parameters of problem (P). Because $p_2(n)$ depends on n, some of these parameters also depend on n:

$$a_1 = p_1\lambda_1 + (1 - p_1) + (1 - \lambda_1)\delta_1 p_1, \qquad a_1 \in (0, 1) \tag{160}$$

$$b_1(n) = (1 - \lambda_1)p_1\big(p_2(n)\alpha + (1 - p_2(n))\sigma\big), \qquad b_1(n) > 0 \tag{161}$$

$$a_2 = (1 - p_1)\lambda_1 + p_1, \qquad a_2 \in (0, 1) \tag{162}$$

$$c_2 = \delta_1(1 - \lambda_1)(1 - p_1), \qquad c_2 \in (0, 1) \tag{163}$$

$$b_2(n) = (1 - p_1)(1 - \lambda_1)(p_2(n)\phi + (1 - p_2(n))\beta) \qquad b_2 > 0. \tag{164}$$

Under assumptions $\delta_1 \longrightarrow 1$ and $\forall n \leq n^*$, $\frac{\partial p_2}{\partial n}(n) \geq m$, Lemma 3 allows to state that $\forall n \in \mathbb{N}$, $u_n^n - v_n^n \geq 0$. Using relations (122) and (123), one gets:

$$\forall n \in \mathbb{N},\ \forall k \leq n,\ u_{k+1}^n = a_1 u_k^n + b_1(n) \tag{165}$$

$$v_{k+1}^n = a_2 v_k^n + c_2 u_k^n + b_2(n). \tag{166}$$

We then proceed by weak induction (the initialization is obtained by assumption $u_0 - v_0 > 0$). Let us then assume that:

$$\forall k \leq n,\ u_k - v_k \geq 0 \tag{167}$$

and let us demonstrate that $u_{n+1} - v_{n+1} \geq 0$.

Because one has $u_k - v_k \geq 0$ for $k \leq n$, one can rewrite expressions (43) and (44) (adapted to problem (P)) as follows:

$$\forall k \leq n,\ u_{k+1} = a_1 u_k + b_1(k) \tag{168}$$

$$v_{k+1} = a_2 v_k + c_2 u_k + b_2(k). \tag{169}$$

In particular, one has for $k = n$:

$$u_{n+1} = a_1 u_n + b_1(n) \tag{170}$$

$$v_{n+1} = a_2 v_n + c_2 u_n + b_2(n). \tag{171}$$

Also because $u_n^{n+1} - v_n^{n+1} \geq 0$ (by Lemma 3, we know that $N_1(n+1) \geq n + 1 \geq n$) and $u_n^n - v_n^n \geq 0$, one has:

$$u_{n+1}^n = a_1 u_n^n + b_1(n) \tag{172}$$

$$v_{n+1}^n = a_2 v_n^n + c_2 u_n^n + b_2(n) \tag{173}$$

and

$$u_{n+1}^{n+1} = a_1 u_n^{n+1} + b_1(n+1) \tag{174}$$

$$v_{n+1}^{n+1} = a_2 v_n^{n+1} + c_2 u_n^{n+1} + b_2(n+1). \tag{175}$$

Therefore, one has:

$$u_{n+1} - u_{n+1}^{n+1} = \left(u_{n+1} - u_{n+1}^n\right) + \left(u_{n+1}^n - u_{n+1}^{n+1}\right) \tag{176}$$

$$= a_1(u_n - u_n^n) + \left(u_{n+1}^n - u_{n+1}^{n+1}\right). \tag{177}$$

The general expressions of u_{n+1}^n and u_{n+1}^{n+1} are given by relations (49) adapted to problems $(P)^n$ and $(P)^{n+1}$:

$$u_{n+1}^n - u_{n+1}^{n+1} = (\mu(n) - \mu(n+1)) - a_1^{n+1}(\mu(n) - \mu(n+1)). \tag{178}$$

Therefore:

$$u_{n+1} - u_{n+1}^{n+1} = a_1(u_n - u_n^n) + (\mu(n) - \mu(n+1))\left(1 - a_1^{n+1}\right). \tag{179}$$

A similar reasoning allows to express $v_{n+1} - v_{n+1}^{n+1}$ as follows:

$$v_{n+1} - v_{n+1}^{n+1} = \left(v_{n+1} - v_{n+1}^n\right) + \left(v_{n+1}^n - v_{n+1}^{n+1}\right) \tag{180}$$

$$= a_2(v_n - v_n^n) + c_2(u_n - u_n^n) + \left(v_{n+1}^n - v_{n+1}^{n+1}\right).$$

The general expressions of v_{n+1}^n and v_{n+1}^{n+1} are given by relations (50) adapted to problems $(P)^n$ and $(P)^{n+1}$:

$$v_{n+1}^n - v_{n+1}^{n+1} = \tag{181}$$
$$(\nu(n) - \nu(n+1))(1 - a_2^{n+1}) - (\mu(n) - \mu(n+1))\rho(a_1^{n+1} - a_2^{n+1}).$$

Combining (180) and (181), one gets:

$$v_{n+1} - v_{n+1}^{n+1} = a_2(v_n - v_n^n) + c_2(u_n - u_n^n) \tag{182}$$
$$+ (\nu(n) - \nu(n+1))(1 - a_2^{n+1}) - (\mu(n) - \mu(n+1))\rho(a_1^{n+1} - a_2^{n+1}).$$

Now combining Eqs. (182) with (179), one gets at the limit $\delta_1 \longrightarrow 1$ (recall that a_1 and $\rho \longrightarrow 1$):

$$\left(u_{n+1} - v_{n+1}\right) - \left(u_{n+1}^{n+1} - v_{n+1}^{n+1}\right) = \tag{183}$$
$$(a_1 - c_2)(u_n - u_n^n) - a_2(v_n - v_n^n)$$
$$+ \Big[(\mu(n) - \mu(n+1)) - (\nu(n) - \nu(n+1))\Big](1 - a_2^{n+1}).$$

The term $(a_1 - c_2)(u_n - u_n^n) - a_2(v_n - v_n^n)$ is equal to:

$$(a_1 - c_2)(u_n - u_n^n) - a_2(v_n - v_n^n) = a_2\left((u_n - v_n) - (u_n^n - v_n^n)\right) \tag{184}$$
$$+(a_1 - c_2 - a_2)(u_n - u_n^n)$$

and it can be shown that when $\delta \longrightarrow 1$, $(a_1 - c_2 - a_2) \longrightarrow 0$. Hence, at the limit, one can write:

$$\left(u_{n+1} - v_{n+1}\right) - \left(u_{n+1}^{n+1} - v_{n+1}^{n+1}\right) = \tag{185}$$
$$a_2\left((u_n - v_n) - (u_n^n - v_n^n)\right)$$
$$+ \Big[(\mu(n) - \mu(n+1)) - (\nu(n) - \nu(n+1))\Big](1 - a_2^{n+1}).$$

We recall the expression of $\nu(n) - \mu(n)$ for problem $(P)^n$:

$$\nu(n) - \mu(n) = p_2(n)(\phi - \alpha) + (1 - p_2(n))(\beta - \sigma) \tag{186}$$

therefore:

$$(\mu(n) - \mu(n+1)) - (\nu(n) - \nu(n+1)) = \left(p_2(n+1) - p_2(n)\right)(\phi - \alpha - \beta + \sigma) \tag{187}$$

and we will denote

$$\imath \equiv (\phi - \alpha - \beta + \sigma). \tag{188}$$

Parameter $\imath$ is non-positive by assumption. We can then rewrite Eq. (185) as:

$$\left(u_{n+1} - v_{n+1}\right) - \left(u_{n+1}^{n+1} - v_{n+1}^{n+1}\right) = a_2\left((u_n - v_n) - (u_n^n - v_n^n)\right) + \imath(p_2(n+1) - p_2(n))(1 - a_2^{n+1}). \tag{189}$$

We also recall that by assumption of the weak induction, for $k \le n$, $u_k - v_k \ge 0$ and we already now that $u_k^k - v_k^k \ge 0$, meaning that relation (189) can be written for all $k \le n$. We then obtain a recursive expression of the sequence $\left((u_n - v_n) - (u_n^n - v_n^n)\right)$ that we solve as follows: $\forall k \le n+1$,

$$\left(u_k - v_k\right) - \left(u_k^k - v_k^k\right) = a_2^k\left[\left(u_0 - v_0\right) - \left(u_0^0 - v_0^0\right)\right] + \imath \sum_{i=0}^{k-1} a_2^i(1 - a_2^{k-i})\left(p_2(k-i) - p_2(k-i-1)\right). \tag{190}$$

Problems $(P)^0$ and P coincide at Iteration 0, implying that $u_0^0 = u_0$ and $v_0^0 = v_0$. Therefore:

$$\forall k \le n+1,\ \left(u_k - v_k\right) = \left(u_k^k - v_k^k\right) + \imath \sum_{i=0}^{k-1} a_2^i(1 - a_2^{k-i})\left(p_2(k-i) - p_2(k-i-1)\right). \tag{191}$$

We know that $\left(u_k^k - v_k^k\right) = f_{\rho,k}(k)$, where function $f_{\rho,k}$, which has been defined in Eq. (56), is adapted to problem $(P)^k$ as follows (recall that a_1, a_2, ρ do not depend on p_2):

$$\forall x \ge 0,\ f_{\rho,k}(x) = a_1^x\left[(u_0 - \mu(k))(1-\rho)\right] + a_2^x\left[\rho(u_0 - \mu(k)) - (v_0 - \nu(k))\right] + (\mu(k) - \nu(k)). \tag{192}$$

We fix now $k \le n+1$ and analyze all terms of Eq. (191). Most importantly, we are interested in bounding $u_k^k - v_k^k$ so as $u_k - v_k$ remains non-negative (which is what we intend to prove). Two cases have to be considered.

- When $N_1(k)$ is finite.
  In that case, by definition of threshold $N_1(k)$ we can write that $f_{\rho,k}(N_1(k)) = 0$. Hence:

$$\left(u_k - v_k\right) = \left(f_{\rho,k}(k) - f_{\rho,k}(N_1(k))\right) + \imath \sum_{i=0}^{k-1} a_2^i(1 - a_2^{k-i})\left(p_2(k-i) - p_2(k-i-1)\right). \tag{193}$$

  We now focus on the term $\left(f_{\rho,k}(k) - f_{\rho,k}(N_1(k))\right)$. By the mean variance theorem, one has:

$$\exists c \in [k, N_1(k)] \text{ such that } \left(f_{\rho,k}(k) - f_{\rho,k}(N_1(k))\right) = f'_{\rho,k}(c)(k - N_1(k)). \tag{194}$$

  The derivative of $f_{\rho,k}$ is expressed as follows:

$$f'_{\rho,k} = \ln(a_1)a_1^x\left[(u_0 - \mu(k))(1-\rho)\right] + \ln(a_2)a_2^x\left[\rho(u_0 - \mu(k)) - (v_0 - \nu(k))\right] \tag{195}$$

  and at the limit $\delta \longrightarrow 1$ (recall that a_1 and $\rho \longrightarrow 1$ and that $(1-\rho)\mu(k) \longrightarrow \mu'(k)$ as shown in Eq. (135)):

$$f'_{\rho,k}(x) = \ln(a_2)a_2^x\left[(u_0 - \mu(k)) - (v_0 - \nu(k))\right]. \tag{196}$$

  A similar analysis at the limit of the second derivative of $f_{\rho,k}$ allows to state that function $f'_{\rho,k}$ is an increasing and negative function. As consequence:

$$\forall c \in [k, N_1(k)],\ f'_{\rho,k}(c) \le f'_{\rho,k}(N_1(k)) = \ln(a_2)a_2^{N_1(k)}\left[(u_0 - \mu(k)) - (v_0 - \nu(k))\right]. \tag{197}$$

  By Lemma 2, we have:

$$a_2^{N_1(k)}\left[(u_0 - \mu(k)) - (v_0 - \nu(k))\right] = \nu(k) - \mu(k) + \mu'(k). \tag{198}$$

  Remark that $\nu(k) - \mu(k) + \mu'(k) \ge 0$ because $N_1(k)$ is finite (Lemma 2). Plugging relation (198) into Eq. (197), we get:

$$\forall c \in [k, N_1(k)],\ f'_{\rho,k}(c) \le \ln(a_2)\left(\nu(k) - \mu(k) + \mu'(k)\right) \tag{199}$$

  and plugging the latter into relation (194) (recall that $k \le N_1(k)$ by Lemma 3):

$$\left(f_{\rho,k}(k) - f_{\rho,k}(N_1(k))\right) \ge \ln(a_2)\left(\nu(k) - \mu(k) + \mu'(k)\right)(k - N_1(k)) \tag{200}$$

  and plugging this into relation (193), one gets:

$$\left(u_k - v_k\right) \ge \ln(a_2)\left(\nu(k) - \mu(k) + \mu'(k)\right)(k - N_1(k)) + \imath \sum_{i=0}^{k-1} a_2^i(1 - a_2^{k-i})\left(p_2(k-i) - p_2(k-i-1)\right). \tag{201}$$

  Relation (157) of the proof of Lemma 3 allows to write:

$$\forall n \in \mathbb{N},\ N_1(k) - k \ge N_1(n=0) > 0 \tag{202}$$

  and getting back to relation (201) (recall that $a_2 < 1$):

$$\left(u_k - v_k\right) \ge -\ln(a_2)\left(\nu(k) - \mu(k) + \mu'(k)\right)N_1(n=0) + \imath \sum_{i=0}^{k-1} a_2^i(1 - a_2^{k-i})\left(p_2(k-i) - p_2(k-i-1)\right). \tag{203}$$

  We have from relations (135) and (139) and from assumptions $\phi - \alpha - \beta + \sigma \le 0$ and $p_1 \ge \frac{1}{2}$:

$$\begin{aligned}\left(\nu(k) - \mu(k) + \mu'(k)\right) &= p_2(k)(\phi - \alpha) + (1 - p_2(k))(\beta - \sigma) \\ &\quad + \left(p_2(k)\alpha + (1 - p_2(k))\sigma\right)\frac{(1 - 2p_1)}{(1 - p_1)} \\ &\ge (\phi - \alpha).\end{aligned} \tag{204}$$

  Hence:

$$\left(u_k - v_k\right) \ge -\ln(a_2)N_1(n=0)(\phi - \alpha) + \imath \sum_{i=0}^{k-1} a_2^i(1 - a_2^{k-i})\left(p_2(k-i) - p_2(k-i-1)\right). \tag{205}$$

  We define now parameter $q_1 > 0$ as follows:

$$q_1 \equiv \frac{\ln(a_2)}{\imath}(1 - a_2)N_1(n=0)(\phi - \alpha). \tag{206}$$

  If we have $\forall l \in \mathbb{N},\ \frac{\partial p_2}{\partial n}(l) \le q_1$, we get from relation (205) (recall that $\imath \le 0$) the following:

$$\begin{aligned}\left(u_k - v_k\right) &\ge -\ln(a_2)N_1(n=0)(\phi - \alpha) + \imath \sum_{i=0}^{k-1} a_2^i(1 - a_2^{k-i})\left(p_2(k-i) - p_2(k-i-1)\right) \\ &\ge -\ln(a_2)N_1(n=0)(\phi - \alpha) + \imath q_1 \sum_{i=0}^{k-1} a_2^i(1 - a_2^{k-i}) \\ &\ge -\ln(a_2)N_1(n=0)(\phi - \alpha) + \imath q_1 \frac{1}{1 - a_2} \\ &= 0.\end{aligned} \tag{207}$$

- When $N_1(k)$ is infinite. Here, the mean value theorem can no longer be invoked to find a lower bound to $u_k^k - v_k^k$. Starting again from Eq. (192), we can write at the limit $\delta_1 \longrightarrow 1$ that:

$$u_k^k - v_k^k = a_2^k\left[(u_0 - \mu(k)) - (v_0 - \nu(k))\right] + (\mu(k) - \nu(k) - \mu'(k)). \tag{208}$$

  The term $(\mu(k) - \nu(k) - \mu'(k))$ is non-negative because $N_1(k)$ is infinite (Lemma 2). Because $\imath \le 0$, $(\mu(l) - \nu(l) - \mu'(l))$ is an

increasing sequence of l that convergences to $p_{2,f}(\phi-\alpha)+(1-p_{2,f})(\beta-\sigma)+\left(p_{2,f}\alpha+(1-p_{2,f})\sigma\right)\frac{(1-2p_1)}{(1-p_1)}>0$. Because the term $a_2^k\left[(u_0-\mu(k))-(v_0-\nu(k))\right]$ is also positive, one will deduce that the sequence $a_2^l\left[(u_0-\mu(l))-(v_0-\nu(l))\right]+(\mu(l)-\nu(l)-\mu'(l))$ has a positive minimal value that we denote by w. In other words, we can write that:

$$u_k^k-v_k^k\geq w>0. \tag{209}$$

At this stage, the same line of reasoning that was carried out for the previous case ($N_1(k)$ finite) will allow to state that when we define now parameter $q_2>0$ as follows:

$$q_2\equiv-\frac{w}{\imath}(1-a_2) \tag{210}$$

and if we impose $\forall l\in\mathbb{N},\ \frac{\partial p_2}{\partial n}(l)\leq q_2$, we get that:

$$\left(u_k-v_k\right)\geq 0. \tag{211}$$

The synthesis of both cases (for $k=n+1$) implies that if we impose that $\forall l\in\mathbb{N},\ \frac{\partial p_2}{\partial n}(l)\leq q$ with q defined as:

$$q\equiv\mathrm{Min}(q_1,q_2) \tag{212}$$

then $\left(u_{n+1}-v_{n+1}\right)\geq 0$. This proves the induction for $k=n+1$. □

A.6. A brief discussion of Theorem 4

Theorem 4 holds only when $m\leq q$. This is always true provided that $\imath$ is small enough. Indeed, an inspection of the components of the expression of m provided in Eq. (154) and q provided in Eq. (212) reveals that:

$$m\longrightarrow_{\imath\longrightarrow 0}0 \text{ and } q\longrightarrow_{\imath\longrightarrow 0}+\infty. \tag{213}$$

A small $\imath$ means that the symmetry premium of the matrix of the stage game is small, which is not too restrictive. Besides, when the final value of p_2 is $p_{2,f}=1$, assumption $p_1\geq\frac{p_{2,f}\phi+(1-p_{2,f})\beta}{p_{2,f}(\phi+\alpha)+(1-p_{2,f})(\beta+\sigma)}$ can be reformulated as:

$$p_1\geq\frac{\phi}{(\phi+\alpha)}. \tag{214}$$

When $\imath$ is small, the right-hand side of (214) gets close to $\frac{1}{2}$, which makes the assumption quite natural.

A.7. The case of a simultaneous decrease of the exploration rates

In this section, we try to give an intuition on how relation (29) extends to the case where both algorithms modify their exploration rate. To that end, we do not analyze the whole dynamics of the game but focus only on the limit of learning where the exploration rate of Algorithm 1 (respectively Algorithm 2) is equal to its limit $\epsilon_{1,f}$ (respectively $\epsilon_{2,f}$). If at the limit, each algorithm still believes that cooperation is dominant, we can rewrite condition $\epsilon_{2,f}<\frac{p_1(\phi+\alpha)-\phi}{p_1(\phi+\alpha-\beta-\sigma)+\beta-\phi}$ as follows:

$$\epsilon_{2,f}<\frac{(1-\epsilon_{1,f})(\phi+\alpha)-\phi}{(1-\epsilon_{1,f})(\phi+\alpha-\beta-\sigma)+\beta-\phi} \tag{215}$$

and the symmetric relation:

$$\epsilon_{1,f}<\frac{(1-\epsilon_{2,f})(\phi+\alpha)-\phi}{(1-\epsilon_{2,f})(\phi+\alpha-\beta-\sigma)+\beta-\phi} \tag{216}$$

which leads to the following relations that induce cooperation at equilibrium:

$$\begin{aligned}&\epsilon_{2,f}(\alpha-\sigma)-\epsilon_{1,f}\epsilon_{2,f}(\phi+\alpha-\beta-\sigma)+\epsilon_{1,f}(\phi+\alpha)-\alpha<0\\&\epsilon_{1,f}(\alpha-\sigma)-\epsilon_{1,f}\epsilon_{2,f}(\phi+\alpha-\beta-\sigma)+\epsilon_{2,f}(\phi+\alpha)-\alpha<0.\end{aligned} \tag{217}$$

These relations define the intersection of the interior of two conics that we denote by $\mathcal{C}$. Fig. 8 gives a graphical representation of $\mathcal{C}$ (recall that $\epsilon_{1,f}$ and $\epsilon_{2,f}$ belong to $[0,1]$). We observe that both final exploration rates have to be small to belong to domain $\mathcal{C}$. This would hence suggest that it suffices that one algorithm explores thoroughly to limit cooperation to occur. However, this result provides only an intuition that has to be proved more rigorously, or at least verified numerically, because of two reasons: first, only a reasoning at the limit of exploration is conducted and the whole dynamics of learning is overlooked. Second, the fact the final exploration rates belong to domain $\mathcal{C}$ is a sufficient, but not a necessary, condition for cooperation to dominate competition on average.

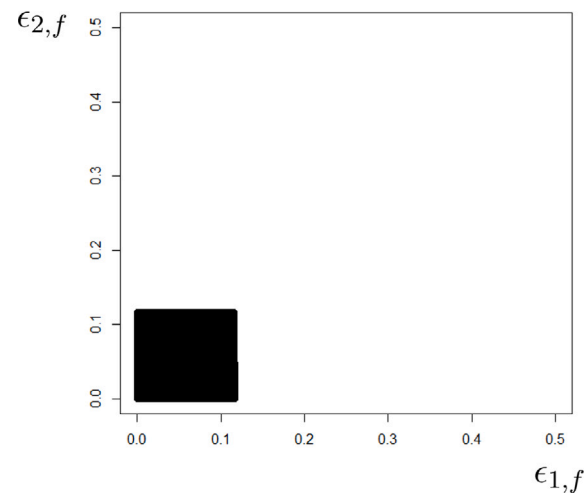

Fig. 8. A graphical representation of domain $\mathcal{C}$.

Appendix B. Robustness checks

In this section, we perform a sensitivity analysis to assess the impact of the Prisoner's Dilemma game's parameters on the extent of cooperation between the stylized Q-learning algorithms analyzed in Section 3.3. In line with the findings of Fig. 3, we focus on the case where algorithms are symmetric and we report on the evolution of the expected values of cooperation and competition in the Q-matrix of Algorithm 1, as well as the probability that the latter cooperates with Iterations n. Exploration decreases as in relationship (30), we consider two final exploration rates 0.15 and 0.25, and we set $1-\gamma=0.8$. Besides, we remind the reader that $\lambda_1=\lambda_2=0.9$, $\delta_1=\delta_2=0.97$, and $N=350$. In Fig. 3, parameters of the game were initially set to $\phi=115$, $\alpha=80$, $\beta=55$, and $\sigma=15$. In this sensitivity, we successively modify the value of each of these parameters individually, resulting in four new simulations whose outcomes are reported in Figs. 9 (sensitivity on ϕ), 10 (sensitivity on α), 11 (sensitivity on β), and 12 (sensitivity on σ). We make sure that the Prisoner's Dilemma condition (1) is always satisfied.

These results are largely in line with our initial theoretical findings and simulations. It appears that decreasing ϕ increases overall the probability to cooperate and the difference between the expected values of cooperation and competition: this is due to the fact that a reduced ϕ decreases the incentive to defect when the competitor cooperates. Second, an increased value of α, which is the immediate profit of cooperation, also increases the probability to cooperate and the expected value of cooperation with respect to defection. Third, a decrease of β has the same effect as this reduces the immediate reward of the Nash equilibrium of the Prisoner's Dilemma. Finally, an increase of σ also has a positive contribution on the propensity of algorithms to cooperate as this reduces the degree of punishment when the opponent defects from a state where both algorithms cooperate. Overall, except for the sensitivity on α (Fig. 10), these results confirm that a thorough exploration limits cooperation. Interestingly, with a high value of α

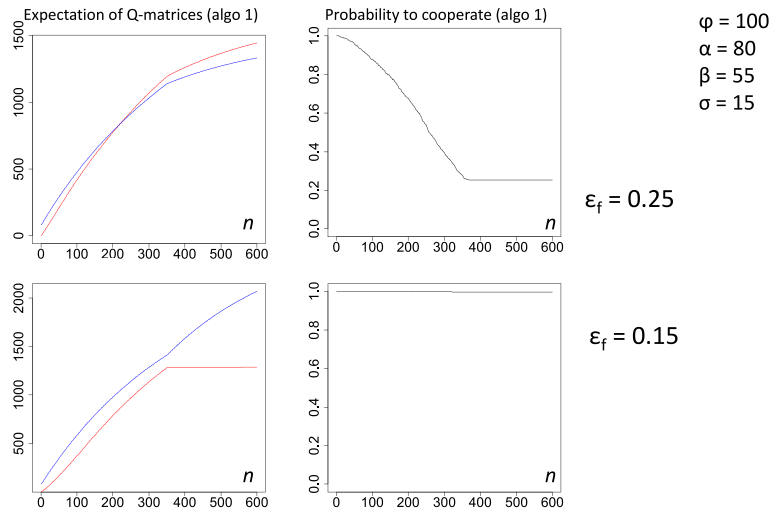

Fig. 9. Left: evolution of the expectation of the Q-values for Algorithm 1 with iterations for cooperation (blue) and competition (red) and for two final exploration rates. Right: evolution of the probability to cooperate with iterations for two final exploration rates. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

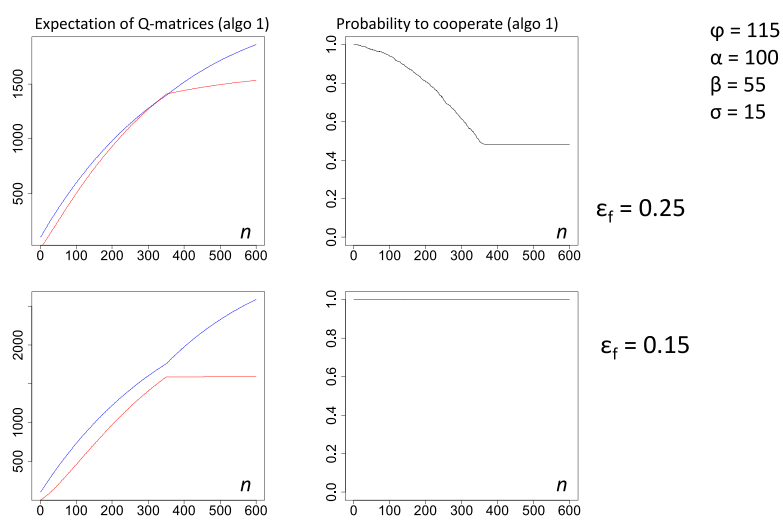

Fig. 10. Left: evolution of the expectation of the Q-values. Right: evolution of the probability to cooperate. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

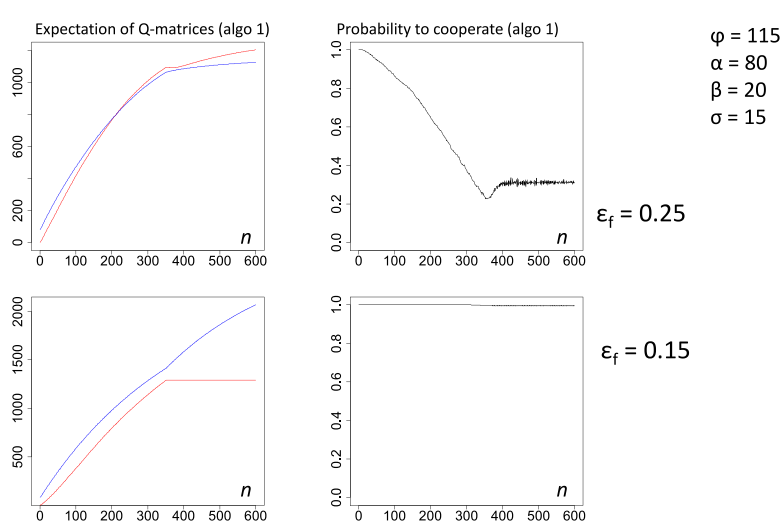

Fig. 11. Left: evolution of the expectation of the Q-values. Right: evolution of the probability to cooperate. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

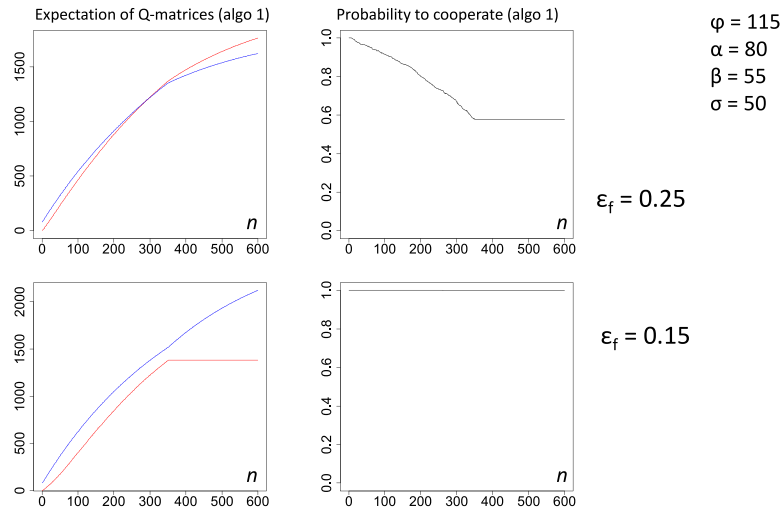

Fig. 12. Left: evolution of the expectation of the Q-values. Right: evolution of the probability to cooperate. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

(which is the immediate reward when both algorithms cooperate), even the rather thorough exploration we consider ($\epsilon_f = 0.25$) is not sufficient for algorithms to learn that competition might, in expectation, yield higher profits than cooperation in a Prisoner's Dilemma. To validate this interpretation, we re-run the sensitivity on parameter α but with a slightly higher value of ϵ_f which we set to 0.3 and verified that cooperation becomes less profitable in expectation than competition, and that algorithms cooperate with almost zero probability at the end of learning.

Appendix C. Description of the reinforcement learning algorithms

C.1. Q-learning

The notions developed in this section are standard. The editing is in part inspired from Abada and Lambin (2023), Section 3.3, to which we refer for further details. Q-learning is a particular and simple case of reinforcement learning where the long-term reward function is approximated by a table, the so-called Q-matrix (or Q-table). The rows of the Q-matrix comprise the states of the system based on which the algorithm will select an action to play. A state can represent the market price (or some aggregated information like demand or supply), past actions played by the algorithm or the competitors, or any direct observation that can be made from the system in which it operates. The algorithm works in perfect monitoring if it can constantly observe all actions that are played by all agents (including competitors, this is possible for example in some online marketplaces where prices and all agents decisions can be tracked in real time) and incorporates them in its modeling of the states. Otherwise, the algorithm works in imperfect monitoring. The columns of the Q-table represent the set of possible actions available to the algorithm. States and actions have to be discretized to fit into the dimensions of the Q-table (these are hyper-parameters set by the user).

The algorithm keeps track of past actions and their rewards directly in the Q-table. More precisely, in a state of the world s, the Q-table estimates for each action a, the long-term expected and discounted optimal revenue accrued starting from state s when the algorithm plays action a. This is similar to the notion of value function in dynamic programming declined over all possible states. The algorithm is iterative and follows two major steps at each episode n. In the first step, the algorithm observes the state of the system and chooses an action to play following a probability distribution $\theta_n(.)$ derived from its current estimation of the Q-table. In the second step, the algorithm observes its reward and updates its Q-matrix as a weighted and discounted average of past and recent payoffs, using the so-called learning rate (that we will define later). The probability distribution $\theta_n(.)$ used to select which action to play will determine the amount of exploration of the algorithm. In the first episodes, the algorithm widely explores by playing actions randomly and observing related payoffs. In subsequent episodes, the algorithm plays its profit-maximizing policy, according to its Q-table, increasingly often. The trade-off between exploration and operations depends on the evolution of the probability distribution $\theta_n(.)$ with episodes n. A wide distribution in the first stages will lead to a better approximation of the value function by the Q-matrix but might drastically increase the convergence time of the algorithm. On the contrary, a narrow distribution concentrated on the optimal value of the Q-table explores less but speeds up the learning process.

Q-learning is quite popular as it has many advantages. First, it is intuitive, simple to implement and requires few parameters: the speed of exploration and the actualization parameters of the Q-matrix (mainly the learning rate and discount factor). Second, Q-learning algorithms do not need to know *a priori* the transition probability function between states of the system when an action is played as it will be learned along with the value function. Therefore, the Q-matrix can be initialized at any value. Finally, the values of the Q-table are interpreted as an estimation of the expected long-term discounted payoffs for each possible action and state. Therefore, when put on the market, it becomes easy to justify why an action will be selected by the algorithm when it sees a given state of the world, increasing thereby explainability of the algorithm. This said, it should nevertheless be noted that like all reinforcement learning algorithms, the estimation of the weights of the Q-matrix can be subject to biases depending on the exploration policy of new states during learning. It also depends on whether and how the algorithm continues to update its Q-table when put on the job.

We now provide a more detailed description of a standard implementation of Q-learning. The various observable states of the system are denoted by $s \in S$ and the various actions of the algorithm by $a \in A$. In state s, if action a is played, the immediate observed reward (or payoff) is denoted by $\pi(s,a)$ and the system moves to state $s'(s,a)$ following, by assumption, a Markov transition probability. The weight of the Q-table is defined as the immediate profit plus the expected maximum future discounted payoff:

$$Q(s,a) = \pi(s,a) + \delta\mathbb{E}\left[\max_{a' \in A} \; Q(s'(s,a),a')\right] \tag{218}$$

and the best action to play in state s to optimize the payoff is: $\text{argmax}_{a\in A} Q(s,a)$. Parameter $\delta \in [0,1)$ is the discount rate. It has to be positive to account for the dynamics of the system and the impact of a decision on future payoffs directly in the Q-matrix.

The calculation of the Q-matrix is usually carried out by iterative approximations. In episode n, the best approximation of the Q-matrix is denoted by Q_n (the matrix can be initialized at zero: $Q_0 = 0$). If state s_n occurs at episode n, the agent decides which action to play according to a distribution $\theta_n(a)$ over A. The simplest exploration policy is epsilon-greedy: at episode n, the optimal decision according to the Q_n-matrix, $a^* = \text{argmax}_{a\in A} Q_n(s_n, a)$, is chosen at probability $1 - \epsilon(n)$:

$$\theta_n(a*) = 1 - \epsilon(n) \tag{219}$$

and the other actions $a \neq a*$ are played with probability $\epsilon(n)$ (a qualitatively equivalent randomization rule allows for selection of the optimal action when the algorithm explores; for consistency with our theoretical illustration and to enhance clarity, we adopt the convention that the optimal action cannot be chosen when algorithms explore):

$$\theta_n(a) = \frac{1}{|A| - 1}\epsilon(n) \text{ for } a \neq a^* \tag{220}$$

with the exploration rate $\epsilon(n)$ decreasing from 1 to zero when $n \longrightarrow +\infty$. Other exploration policies can be used like the Boltzmann's:

$$\theta_n(a) = \frac{\exp\left(\frac{Q_n(s_n,a)}{\epsilon(n)}\right)}{\sum_{a'} \exp\left(\frac{Q_n(s_n,a')}{\epsilon(n)}\right)}, \quad \forall a \in A. \tag{221}$$

The Q-matrix is updated as follows: in learning episode n and state s_n, the action a_n is sampled via distribution $\theta_n(.)$, the algorithm sees its immediate payoff $\pi(s_n, a_n)$ and to which state $s_{n+1}(s_n, a_n)$ (denoted by s_{n+1} for simplicity) the system moves once all algorithms played. The new Q-matrix is updated as:

$$\begin{aligned} \text{if } \ s = s_n \text{ and } a = a_n\text{:} \ \ Q_{n+1}(s_n, a_n) \quad &= \lambda Q_n(s_n, a_n) \\ &+ (1-\lambda)\left(\pi(s_n, a_n) + \delta \max_{a'\in A} Q_n(s_{n+1}, a')\right) \end{aligned} \tag{222}$$

$$\text{otherwise: } \ Q_{n+1}(s, a) \quad = Q_n(s, a).$$

The updated Q-matrix is a convex combination of two terms. The firs term is its previous value. The second term is the sum of the immediate payoff and the optimal discounted future profit as estimated by the previous Q-matrix. The learning dynamics is controlled by the so-called learning rate $1 - \lambda$: the smaller λ, the larger the weight of the recent (discounted) profit in the updating process. Besides, some memory can be incorporated in learning as a state s_n can encompass current and past market observations (encountered in the previous episodes).

Several convergence results of Q-learning exist in the literature under standard assumptions on stationarity, on the dynamics of the states (that have to follow a Markovian process), and the dynamics of learning. However, in a competing multi-agent framework, each algorithm learns in a non-stationary environment when training is carried out independently, as any action taken by an algorithm modifies the environment for all the others (Lowe et al., 2017). In addition, simultaneously trained (Q-)learning algorithms cannot map the transition probability between states with their actions. This has two consequences: the convergence (if any) time drastically increases on the one hand and no general theoretical convergence results hold on the other hand. To the best of our knowledge, the only results apply to stochastic games in Q-matrices with perfect monitoring and where each algorithm keeps track of its own Q-table, but also an approximation of the ones of the others.

C.2. Actor–critic algorithms

In a critic-only method, the algorithm's goal is to approximate the value function as in tabular Q-learning but by deploying more sophisticated methods, like neural networks, without explicitly optimizing the action policy. Adapted optimization techniques are subsequently deployed to estimate the best policy to play for a given state of the system. On the contrary, an actor-only method tries to maximize the expected payoff by optimizing the policy with which the algorithm selects its actions, without explicitly keeping track of the value function. A policy $\theta(s) \in \mathcal{P}(s) = \{(\theta_1(s), \theta_2(s), \dots \theta_{|A|}(s)) \in [0,1]^{|A|} \text{ with } \sum_j \theta_j(s) = 1\}$ is a probability distribution over the set of possible actions A, for a given state s. The policy is state-dependent and to ease notation, the algorithm's parameters are concatenated into a multi-dimensional variable ϕ. This implies that the policy is in fact function of the state s and parameter ϕ: $\theta(s, \phi) \in \mathcal{P}(s)$. The algorithm reinforces its learning by iteratively updating the value of ϕ given new observations of states s and rewards, with the purpose of maximizing its returns. Updating parameters ϕ is done using adapted gradient methods (also called policy gradient (PG) algorithms). Numerically, policy gradient methods can behave poorly as they may exhibit high fluctuations or variance in the estimation of the gradient of the objective with respect to ϕ. This may increase the convergence time and deteriorate the performances of the algorithms after learning. This effect is further enhanced in a multi-agent setting where the reward function does not only depend on the algorithm's policy, but also on the others' and where the numerical estimation of the policy gradient will suffer from some imprecision as the other algorithm's policies cannot be fixed when learning is simultaneous. Therefore, as with Q-learning, non-stationarity of the environment as experienced by each algorithm can lower performances and increase convergence time of actor and critic methods. It is also the reason for the absence (to the best of our knowledge) of general convergence results for multi-agent learning.

Actor–Critic algorithms aim at exploiting the advantages of actor-only and critic-only methods. They simultaneously combine, teach, and operate an actor *and* a critic algorithms. The critic approximates the value function following a reinforcement learning logic where payoffs are generated via the policy estimated by the actor. In turn, the approximation of the value function will be used by the actor to update its parameters via the estimation of the policy gradient. One has then to keep track of two inter-dependent neural networks which requires more calculation time and power, but with the advantage that Actor–Critic methods yield more desirable numerical properties (mainly due to variance reduction of policy gradients), better performances, and reduced convergence time than actor-only and critic-only methods.

Like for Q-learning, actor-only, and critic-only methods, convergence results in a multi-agent framework are not available for actor–critic algorithms, also because of a lack of stationarity of the environment as perceived by each algorithm individually. In terms of explainability, Actor–Critic methods are much more complex than Q-learning, especially considering the fact that both actor and critic modules are generally modeled using multi-layer neural networks. The value function can be analyzed via the critic network but linking it to the input parameters is not straightforward as this dependence is the result of linear and non-linear (via the activation functions) combinations of the values of many nodes of the network. The choice of action is left to the actor model, which complicates even further the interpretation of the output as its direct link with the expected payoff is lost (the dependence between actor and critic networks is not explicit). As a consequence, Actor–Critic methods are often considered as black boxes by their users. In turn, thanks to this increased sophistication, they usually perform much better than Q-learning for standard problems.

In this paper, we implement a particular instance of a Continuous Actor–Critic algorithm inspired from Wang et al. (2016). The input layer of the actor and critic networks comprise the observable states of the market, in our case the spot price at the previous iteration (treated as a continuous data). The output layer of the actor network contains two real numbers η and $\omega \geq 0$ that will be used to sample the action to play using a normal distribution of expectation η and variance ω, $\mathcal{N}(\eta, \omega)$, projected over the action space via a sigmoid (logistic) function. The output layer of the critic represents the optimal long-term expected profit (exactly as for Q-learning), that we also call the value

function $V(.)$. Both actor and critic networks are updated iteratively as follows: at Iteration n, a batch of B experiences are simulated by sampling actions $a_1, a_2, \dots, a_B$ from the actor (as estimated at Iteration n) and immediate rewards $\pi_1, \pi_2, \dots, \pi_B$ and states $s_1, s_2, \dots, s_B$ are observed. For each experience $b = 1, \dots, B$, the advantage adv_b is then calculated as the difference between two terms. The first term is the sum of the immediate observed reward π_b and the discounted future value as estimated by the critic (at Iteration n): $\delta V^n(s_{b+1})$ (by convention, we set $V^n(s_{B+1}) = 0$). We recall that $\delta \in [0, 1)$ is the discount rate. The second term is the estimation of the value function provided by the critic at experience b: $V^n(s_b)$. The critic will then be updated at Iteration $n+1$ by minimizing the mean (across the B experiments) square of the advantages. In other words, we want the critic to be the most realistic in its prediction of the value function. The updating of the actor network is carried out by minimizing the following loss function: $-\sum_{i=1}^{B} \ln(p_i) adv_i$, where p_i denotes the probability of action a_i, that we derive from the expression of the normal distribution $\mathcal{N}(\eta, \omega)$. When minimizing this loss function, the actor network will hence strive to tune parameters η and ω so as to put more weight on the actions that generated high advantages. Exploration is hence quite natural: in the early stages of learning, the variance of the normal distribution ω is set at a quite high value leading to a wide sampling of actions to play. In subsequent periods, ω is reduced to concentrate more on actions that generate high rewards. Relevant to our research, we remark that in comparison with Q-learning, the exploration policy is endogenous to the learning process.

The particular implementation of CAC in the base case of this paper is as follows:

- The actor network has dimensions [1,256,2]: input is the price, one hidden layer of size 256 and two outputs — one for the mean η and one for the variance ω of the normal distribution. The activation function of the first layer is Relu and the two activation functions of the second layer are a scaled tanh (for η) and a softplus (for ω).
- The critic network has dimensions [1,256,1]: input is the price, one hidden layer of size 256 and one output corresponding to the value estimate. The activation function is Relu.

- We set hyper-parameter B to 100 and we keep a one-period memory of the states, a value that is sufficient to trigger seeming collusion for simple Q-learning algorithms (as in Calvano, Calzolari, Denicolò, and Pastorello (2020) and Abada and Lambin (2023)). The discount rate is 0.98. Learning is carried out during 50,000 iterations. The minimization of the loss functions of the actor and critic is performed via a gradient descent. The learning rate is 0.0002.

C.3. The reinforce algorithm

The reinforce algorithm (Williams, 1992) is an actor-based reinforcement learning method that strives to optimize the best policy without necessarily estimating the value function as in Q-learning. Using the notation introduced in Appendix C.2 where states are denoted by s, actions by a, the algorithm's hyper-parameters by a multi-dimensional vector ϕ, and the state-dependent policy by $\theta(s, \phi) \in [0, 1]^{|A|}$, the updating of the policy is carried out as follows: at Iteration n and state s_n and if action a_n is played, we simulate B actions $a_{n+1}, a_{n+2}, \dots, a_{n+B}$ and collect the dynamics of the states $s_{n+1}, s_{n+2}, \dots, s_{n+B}$ along with the corresponding rewards $\pi_{n+1}, \pi_{n+2}, \dots, \pi_{n+B}$. This is undertaken under the current policy $\theta(., \phi_n)$. The future discounted profit is calculated as:

$$G = \sum_{k=1}^{B} \delta^{k-1} \pi_{n+k} \tag{223}$$

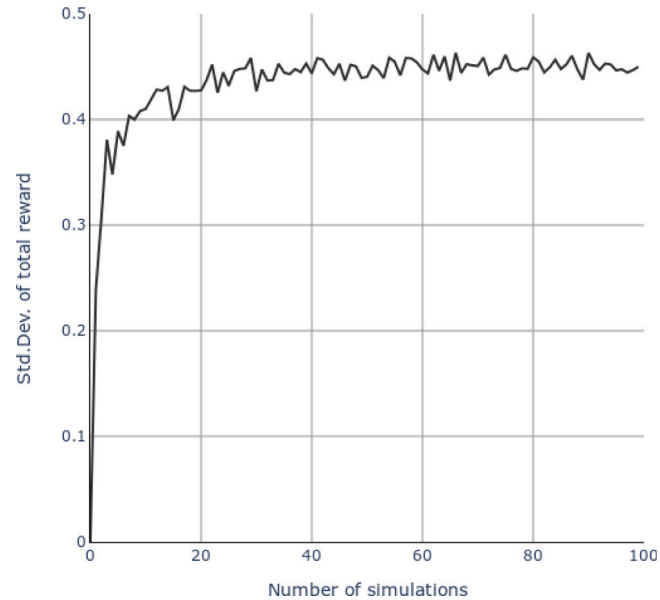

Fig. 13. Estimation of the standard deviation of the total profit at the end of learning with two symmetric Q-learners for samples of varying sizes.

and the updating of ϕ_n is (assuming that the policy never assigns zero to any action):

$$\phi_{n+1} = \phi_n + \alpha\ G\ \partial_\phi \Big(\ln \big(\theta(s_n, \phi_n)(a_n) \big) \Big). \tag{224}$$

Parameter α is the learning rate and ∂_ϕ is the gradient operator with respect to ϕ. The term $\theta(s_n, \phi_n)(a_n)$ is the probability to play action a_n according to the current distribution $\theta(s_n, \phi_n)$. Eq. (224) can be rewritten as:

$$\phi_{n+1} = \phi_n + \alpha\ G\ \frac{\partial_\phi \Big(\theta(s_n, \phi_n)(a_n) \Big)}{\theta(s_n, \phi_n)(a_n)} \tag{225}$$

which provides an understanding of what the algorithm does. If $G > 0$, meaning that the future discounted reward is positive under current policy, then the algorithm's hyper-parameters are *reinforced* by taking a step in the direction of the policy's gradient with the purpose of increasing the probability to play the high rewarding actions. The division by $\theta(s_n, \phi_n)(a_n)$ is intended to make the step of the gradient ascent higher if the reward G is positive and the probability of playing action a_n was low.

The particular implementation of the Reinforce algorithm is as follows: The learning rate is 10^{-4} and the discount rate is 0.98. We set hyper-parameter B to 100 and we keep a one-period memory of the states. Learning is carried out during 10,000 iterations.

C.4. The Exp3 algorithm

The Exp3 algorithm (Lattimore & Szepesvári, 2020) is a simple policy-based reinforcement learning approach that overlooks the representation of states. The algorithm is adapted to dynamic learning following the procedure of den Boer et al. (2022). At Iteration n, we denote by $\theta^n = (\theta_1^n, \theta_2^n, \dots, \theta_{|A|}^n) \in [0, 1]^{|A|}$ the policy followed by the algorithm. At Iteration n, a set of B simulations following distribution θ^n will generate actions $z^1, z^2, \dots, z^B$ with rewards $\pi^1, \pi^2, \dots \pi^n$, which translate into a past discounted reward for each action a_i with $i \in \{1, 2, \dots, |A|\}$:

$$\Pi_i^n = \sum_{k=1}^{B} \delta^{k-1} \pi^k \mathbb{1}_{z^k = a_i}. \tag{226}$$

We define weights w_i^n recursively as:

$$w_i^{n+1} = w_i^n \exp\left(\frac{\eta \Pi_i^n}{\theta_i^n |A|}\right) \tag{227}$$

and the updating of the policy is as follows:

$$\forall i \in \{1, 2, \dots, |A|\},\ \theta_i^n = (1-\eta)\frac{w_i^n}{\sum_j w_j^n} + \eta\frac{1}{|A|}, \tag{228}$$

which is a convex combination between the policy that assigns more weight to the highest rewarding actions and the uniform distribution. Hyper-parameter $\nu > 0$ is set to 10^{-4} and the discount rate is 0.995. Learning is carried out during 10,000 iterations and $B = 100$.

Appendix D. Complementary analysis of simulation results

D.1. Convergence of the sample mean estimate

Fig. 13 reports on the estimation of the standard deviation of the total profit at the end of learning when both Agents use Q-learning (y-axis). The estimation is based on samples of varying sizes, that we report on the x-axis. The Figure highlights that this estimation has converged for 100 samples, confirming that this number of simulations is sufficient to grasp the variations of the results.

References

Abada, I., & Lambin, X. (2023). Artificial intelligence: Can seemingly collusive outcomes be avoided? *Management Science*, *69*(9), 5042–5065.

Alfonso-Sánchez, S., Solano, J., Correa-Bahnsen, A., Sendova, K. P., & Bravo, C. (2024). Optimizing credit limit adjustments under adversarial goals using reinforcement learning. *European Journal of Operational Research*, *315*, 802–817.

Asker, J., Fershtman, C., & Pakes, A. (2022). Artificial intelligence, algorithm design, and pricing. *AEA Papers and Proceedings*, *112*, 452–456. http://dx.doi.org/10.1257/pandp.20221059.

Assad, S., Clark, R., Ershov, D., & Xu, L. (2024). Algorithmic pricing and competition: Empirical evidence from the German retail gasoline market. *Journal of Political Economy*, *132*, 723–771.

Autorité de la Concurrence, & Bundeskartellamt (2019). Algorithms and competition november. URL www.bundeskartellamt.de.

Baker, R. S., & Hawn, A. (2022). Algorithmic bias in education. *International Journal of Artificial Intelligence in Education*, *32*, 1052–1092. http://dx.doi.org/10.1007/s40593-021-00285-9.

Banchio, M., & Mantegazza, G. (2022). Adaptive algorithms and collusion via coupling. arXiv preprint arXiv 05946.

Banerjee, D., & Sen, S. (2007). Reaching pareto-optimality in prisoner's dilemma using conditional joint action learning. *Autonomous Agents and Multi-Agent Systems*, *15*, 91–108. http://dx.doi.org/10.1007/s10458-007-0020-8.

Beikmohammadi, A. (2022). NARS vs. Reinforcement learning: ONA vs. Q-learning. arXiv preprint arXiv:2212.12517.

den Boer, A. V., Meylahn, J. M., & Schinkel, M. P. (2022). Artificial collusion: Examining supracompetitive pricing by Q-learning algorithms. Amsterdam Law School Research Paper No. 2022-25.

Bramante, V. S., Calvano, E., Calzolari, G., & Shaefer, M. (2022). Algorithms in the wild: Experimental evidence from an online marketplace. EODS working paper.

Brown, Z. Y., & MacKay, A. (2021). *Competition in pricing algorithms*: *Technical report*, National Bureau of Economic Research.

Burka, D., Puppe, C., Szepesváry, L., & Tasnádi, A. (2022). Voting: A machine learning approach. *European Journal of Operational Research*, *299*, 1003–1017. http://dx.doi.org/10.1016/J.EJOR.2021.10.005.

Calvano, E., Calzolari, G., Denicolò, V., Harrington, J. E., & Pastorello, S. (2020). Protecting consumers from collusive prices due to AI. *Science*, *370*(6520).

Calvano, E., Calzolari, G., Denicolò, V., & Pastorello, S. (2020). Artificial intelligence, algorithmic pricing, and collusion. *American Economic Review*, *110*, 3267–3297. http://dx.doi.org/10.1257/aer.20190623.

Chapman, A. C., Leslie, D. S., Rogers, A., & Jennings, N. R. (2013). Convergent learning algorithms for unknown reward games. *SIAM Journal on Control and Optimization*, *51*, 3154–3180. http://dx.doi.org/10.1137/120893501.

Chen, Y., Calabrese, R., & Martin-Barragan, B. (2024). Interpretable machine learning for imbalanced credit scoring datasets. *European Journal of Operational Research*, *312*, 357–372. http://dx.doi.org/10.1016/j.ejor.2023.06.036.

Chen, S., Guo, Z., & Zhao, X. (2021). Predicting mortgage early delinquency with machine learning methods. *European Journal of Operational Research*, *290*, 358–372. http://dx.doi.org/10.1016/J.EJOR.2020.07.058.

Chen, L., Mislove, A., & Wilson, C. (2016). An empirical analysis of algorithmic pricing on amazon marketplace. In *Proceedings of the 25th international conference on world wide web* (pp. 1339–1349).

Chuchro, R., & Gupta, D. (2017). Game playing with deep Q-learning using OpenAI Gym. Semantic Scholar.

Claus, C., & Boutilier, C. (1998). Dynamics of reinforcement learning in cooperative multiagent systems. In *Proceedings of the national conference on artificial intelligence*.

Concorrència, A. D. (2019). Digital ecosystems, big data and algorithms issues paper. *Autoridade Da Concorréncia*, (July), URL http://www.concorrencia.pt/vEN/Estudos{_}e{_}Publicacoes/Estudos{_}Economicos/Other/Pages/Digital-Ecosystems-Executive-Summary{_}.aspx?lst=1.

Connor, J. M. (2003). Private international cartels: Effectiveness, welfare, and anticartel enforcement. SSRN working papers.

Cooper, W. L., de Mello, T. H., & Kleywegt, A. J. (2015). Learning and pricing with models that do not explicitly incorporate competition. *Operations Research*, *63*, 86–103. http://dx.doi.org/10.1287/opre.2014.1341.

Corredera, A., & Ruiz, C. (2023). Prescriptive selection of machine learning hyperparameters with applications in power markets: Retailer's optimal trading. *European Journal of Operational Research*, *306*, 370–388. http://dx.doi.org/10.1016/J.EJOR.2022.11.011.

Cowgill, B., & Tucker, C. E. (2019). Economics, fairness and algorithmic bias. *Journal of Economic Perspectives*, preparation for.

Crandall, J. W., Oudah, M., Tennom, Ishowo-Oloko, F., Abdallah, S., Bonnefon, J. F., et al. (2017). Cooperating with machines. Toulouse School of Economics, Working Papers.

Crandall, J. W., Oudah, M., Tennom, Ishowo-Oloko, F., Abdallah, S., Bonnefon, J.-F., et al. (2018). Cooperating with machines. *Nature Communications*, *9*, 233. http://dx.doi.org/10.1038/s41467-017-02597-8.

Csató, L. (2021). *Tournament design: How operations research can improve sports rules*. Palgrave Pivots in Sports Economics. Palgrave Macmillan, Cham, Switzerland.

Dehaybe, H., Catanzaro, D., & Chevalier, P. (2024). Deep reinforcement learning for inventory optimization with non-stationary uncertain demand. *European Journal of Operational Research*, *314*, 433–445. http://dx.doi.org/10.1016/j.ejor.2023.10.007.

Dieter, P., Caron, M., & Schryen, G. (2023). Integrating driver behavior into last-mile delivery routing: Combining machine learning and optimization in a hybrid decision support framework. *European Journal of Operational Research*, *311*, 283–300. http://dx.doi.org/10.1016/j.ejor.2023.04.043.

Dogru, O., Velswamy, K., & Huang, B. (2021). Actor–critic reinforcement learning and application in developing computer-vision-based interface tracking. *Engineering*, *7*, 1248–1261. http://dx.doi.org/10.1016/j.eng.2021.04.027.

Dolgopolov, A. (2024). Reinforcement learning in a prisoner's dilemma. *Games and Economic Behavior*, *144*, 84–103. http://dx.doi.org/10.1016/j.geb.2024.01.004.

Doya, K. (2000). Reinforcement learning in continuous time and space. *Neural Computation*, *12*, 219–245. http://dx.doi.org/10.1162/089976600300015961.

Dressel, J., & Farid, H. (2021). The dangers of risk prediction in the criminal justice system. *MIT Case Studies in Social and Ethical Responsibilities of Computing*, http://dx.doi.org/10.21428/2c646de5.f5896f9f.

Epivent, A., & Lambin, X. (2024). On algorithmic collusion and reward–punishment schemes. *Economics Letters*, *237*, Article 111661.

Gajjar, G. R., Khaparde, S. A., Nagaraju, P., & Soman, S. A. (2002). Application of actor-critic learning algorithm for optimal bidding problem of a genco. *IEEE Power Engineering Review*, *22*, http://dx.doi.org/10.1109/MPER.2002.4311813, 55–55.

Gillis, T. B., & Spiess, J. L. (2019). Big data and discrimination. *The University of Chicago Law Review*, *86*(2), 459–488.

Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., et al. (2018). Soft actor-critic algorithms and applications. arXiv preprint arXiv:1812.05905.

Hansen, K. T., Misra, K., & Pai, M. M. (2021). Frontiers: Algorithmic collusion: Supra-competitive prices via independent algorithms. *Marketing Science*, *40*(1), 1–12.

Hettich, M. (2021). Algorithmic collusion: Insights from deep learning. Available at SSRN 3785966.

Hoffmann, A. L. (2019). Where fairness fails: Data, algorithms, and the limits of antidiscrimination discourse. *Information, Communication & Society*, *22*, 900–915. http://dx.doi.org/10.1080/1369118X.2019.1573912.

Hu, J., Wellman, M. P., et al. (1998). Multiagent reinforcement learning: Theoretical framework and an algorithm. *ICML*, *98*, 242–250.

Jin, J., Cui, T., Bai, R., & Qu, R. (2024). Container port truck dispatching optimization using Real2Sim based deep reinforcement learning. *European Journal of Operational Research*, *315*(1), 161–175.

Kaplow, L. (2013). *Competition policy and price fixing*. Princeton University Press, http://dx.doi.org/10.1515/9781400846078.

Katsafados, A. G., Leledakis, G. N., Pyrgiotakis, E. G., Androutsopoulos, I., & Fergadiotis, M. (2024). Machine learning in bank merger prediction: A text-based approach. *European Journal of Operational Research*, *312*, 783–797. http://dx.doi.org/10.1016/j.ejor.2023.07.039.

Klein, T. (2021). Autonomous algorithmic collusion: Q-learning under sequential pricing. *Rand Journal of Economics*, *52*(3), 538–558. http://dx.doi.org/10.1111/1756-2171.12383, URL https://onlinelibrary.wiley.com/doi/abs/10.1111/1756-2171.12383.

Kormelink, J. G., Drugan, M. M., & Wiering, M. A. (2018). Exploration methods for connectionist Q-learning in Bomberman. In *Proceedings of the 10th international conference on agents and artificial intelligence* (pp. 355–362). SCITEPRESS - Science and Technology Publications, http://dx.doi.org/10.5220/0006556403550362.

Lattimore, T., & Szepesvári, C. (2020). *Bandit algorithms*. Cambridge University Press, http://dx.doi.org/10.1017/9781108571401.

Levenstein, M. C., & Suslow, V. Y. (2012). Cartels and collusion-empirical evidence. Ross School of Business Paper, No. 1182.

Littman, M. L. (1994). Markov games as a framework for multi-agent reinforcement learning. In *ICML*.

Loots, T., & denBoer, A. V. (2023). Data-driven collusion and competition in a pricing duopoly with multinomial logit demand. *Production and Operations Management*, *32*(4), 1169–1186.

Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., & Mordatch, I. (2017). Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in neural information processing systems*.

Mailath, G. J., Samuelson, L., et al. (2006). *Repeated games and reputations: Long-run relationships*. Oxford University Press.

Maruta, T. (1997). On the relationship between risk-dominance and stochastic stability. *Games and Economic Behavior*, *19*(2), 221–234.

McHale, I. G., & Holmes, B. (2023). Estimating transfer fees of professional footballers using advanced performance metrics and machine learning. *European Journal of Operational Research*, *306*, 389–399. http://dx.doi.org/10.1016/J.EJOR.2022.06.033.

Meylahn, J. M., & den Boer, A. V. (2022). Learning to collude in a pricing duopoly. *Manufacturing & Service Operations Management*, http://dx.doi.org/10.1287/msom.2021.1074.

Meylahn, J. M., & Janssen, L. (2022). Limiting dynamics for Q-learning with memory one in symmetric two-player, two-action games. *Complexity*, *2022*, 1–20.

Miklós-Thal, J., & Tucker, C. (2019). Collusion by algorithm: Does better demand prediction facilitate coordination between sellers? *Management Science*, *65*, 1552–1561. http://dx.doi.org/10.1287/mnsc.2019.3287.

Müller, D., Müller, M. G., Kress, D., & Pesch, E. (2022). An algorithm selection approach for the flexible job shop scheduling problem: Choosing constraint programming solvers through machine learning. *European Journal of Operational Research*, *302*, 874–891. http://dx.doi.org/10.1016/j.ejor.2022.01.034.

Panch, T., Mattie, H., & Atun, R. (2019). Artificial intelligence and algorithmic bias: Implications for health systems. *Journal of Global Health*, *9*, http://dx.doi.org/10.7189/jogh.09.020318.

Schmidt, D., Shupp, R., Walker, J. M., & Ostrom, E. (2003). Playing safe in coordination games:: The roles of risk dominance, payoff dominance, and history of play. *Games and Economic Behavior*, *42*(2), 281–299.

Sigrist, F., & Leuenberger, N. (2023). Machine learning for corporate default risk: Multi-period prediction, frailty correlation, loan portfolios, and tail probabilities. *European Journal of Operational Research*, *305*, 1390–1406. http://dx.doi.org/10.1016/j.ejor.2022.06.035.

Tedrake, R., Zhang, T. W., & Seung, H. S. (2004). Stochastic policy gradient reinforcement learning on a simple 3D biped. In *Proceedings of the IEEE International Conference on Intelligent Robots and Systems (IROS)*: *vol. 3*, (pp. 2849–2854). Sendai, Japan.

Tsamados, A., Aggarwal, N., Cowls, J., Morley, J., Roberts, H., Taddeo, M., et al. (2021). The ethics of algorithms: Key problems and solutions. *AI & SOCIETY*, http://dx.doi.org/10.1007/s00146-021-01154-8.

Waltman, L., & Kaymak, U. (2008). Q-learning agents in a cournot oligopoly model. *Journal of Economic Dynamics & Control*, *32*(10), 3275–3293.

Wang, Z., Bapst, V., Heess, N., Mnih, V., Munos, R., Kavukcuoglu, K., et al. (2016). Sample efficient actor-critic with experience replay. arXiv preprint arXiv:1611.01224.

Wang, Y., & Pavel, L. (2014). A modified Q-learning algorithm for potential games. In *IFAC proceedings volumes (IFAC-papersOnline)*. http://dx.doi.org/10.3182/20140824-6-za-1003.02646.

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, *8*, 229–256. http://dx.doi.org/10.1007/BF00992696.

Xiong, Y., Liu, Y., Qian, Y., Jiang, Y., Chai, Y., & Ling, H. (2024). Review-based recommendation under preference uncertainty: An asymmetric deep learning framework. *European Journal of Operational Research*, *316*(3), 1044–1057. http://dx.doi.org/10.1016/j.ejor.2024.01.042.

Xu, J., Liu, B., Zhao, X., & Wang, X.-L. (2024). Online reinforcement learning for condition-based group maintenance using factored Markov decision processes. *European Journal of Operational Research*, *315*(1), 176–190.

Yilmaz, D., & Büyüktahtakın, İ. E. (2024). An expandable machine learning-optimization framework to sequential decision-making. *European Journal of Operational Research*, *314*, 280–296. http://dx.doi.org/10.1016/j.ejor.2023.10.045.